

Theoretical Study on Multi-objective Heuristic Search

SHAWN SKYLER, Ben-Gurion University of the Negev, Israel

DOR ATZMON, Bar-Ilan University, Israel

ARIEL FELNER, Ben-Gurion University of the Negev, Israel

SHAHAF SHPERBERG, Ben-Gurion University of the Negev, Israel

OREN SALZMAN, Technion, Israel

CARLOS HERNÁNDEZ ULLOA, Universidad San Sebastian, Chile

SVEN KOENIG, University of California, Irvine, United States

Search in graphs is a core problem in artificial intelligence. Classical optimal heuristic search algorithms, such as A*, assume a single cost on each edge and return a single least-cost path. Many real-world domains, however, involve several conflicting objectives, such as travel time and risk in hazardous material transport, or energy and distance in robotic planning. In such *Multi-Objective Search* problems, each edge is associated with a cost vector, and the aim is to return the entire Pareto-optimal frontier of non-dominated paths.

This paper develops a general theory and a set of practical algorithms for heuristic search in the multi-objective setting. We introduce MOS-A*, a unified best-first search framework parameterized by an ordering function and tie-breaking rule. We provide a principled classification of states and nodes into must-expand, maybe-expand, and never-expand classes, and show that this taxonomy precisely characterizes which nodes must be generated and expanded to discover and certify all Pareto-optimal solutions. The analysis reveals subtle differences between state-based and node-based reasoning, yielding new insights into the role of ordering functions.

Building on this theory, we propose two novel ordering functions, Min and Max. For these objectives in the bi-objective case (with two objectives), we present constant-time dominance-checking schemes. These schemes extend previous work on lexicographic ordering to alternative orderings and dramatically reduce the overhead of dominance checks. Experiments on benchmark road networks demonstrate that different ordering functions discover the Pareto-optimal frontier at markedly different rates, and that constant-time dominance checking makes Min and Max competitive with traditional lexicographic methods. The results highlight trade-offs between early discovery of extreme solutions and balanced coverage of the frontier and provide guidance for choosing an appropriate ordering for anytime Multi-Objective Search applications.

JAIR Associate Editor: Insert JAIR AE Name

JAIR Reference Format:

Shawn Skyler, Dor Atzmon, Ariel Felner, Shahaf Shperberg, Oren Salzman, Carlos Hernández Ulloa, and Sven Koenig. 2026. Theoretical Study on Multi-objective Heuristic Search. *Journal of Artificial Intelligence Research* 4, Article 6 (August 2026), 34 pages. DOI: [10.1613/jair.1.xxxxx](https://doi.org/10.1613/jair.1.xxxxx)

Authors' Contact Information: Shawn Skyler, shawn@post.bgu.ac.il, Ben-Gurion University of the Negev, Israel; Dor Atzmon, dor.atzmon@biu.ac.il, Bar-Ilan University, Israel; Ariel Felner, felner@bgu.ac.il, Ben-Gurion University of the Negev, Israel; Shahaf Shperberg, shperbsh@bgu.ac.il, Ben-Gurion University of the Negev, Israel; Oren Salzman, osalzman@cs.technion.ac.il, Technion, Israel; Carlos Hernández Ulloa, carlos.hernandez@uss.cl, Universidad San Sebastian, Chile; Sven Koenig, helpful.sven@gmail.com, University of California, Irvine, United States.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2026 Copyright held by the owner/author(s).

DOI: [10.1613/jair.1.xxxxx](https://doi.org/10.1613/jair.1.xxxxx)

1 Introduction and Overview

Recent advances in heuristic search have substantially improved the efficiency of finding optimal paths in large graphs with edges annotated with scalar or vector-valued costs. In the classical setting of *Single-Objective Search* (SOS), algorithms such as A* (Hart et al. 1968) aim to identify a least-cost path from a start state to a goal state, optimizing a single cost criterion, e.g., distance, time, or energy. These algorithms, when combined with consistent heuristics, guarantee optimality and efficiency (Dechter and Pearl 1985) and have been widely adopted across planning, robotics, and network routing.

However, real-world decision-making problems often involve multiple objectives that must be optimized simultaneously. This has led to the development of the *Multi-Objective Search* (MOS) setting, in which each edge is labeled with a k -dimensional cost vector, and the task is to compute all *non-dominated solutions* (see definition below) in the Pareto sense, namely, the *Pareto-optimal frontier* (POF). Applications of MOS span diverse domains, including planning energy-efficient and low-cost powerline routes, optimizing inspection plans for autonomous robots (Bachmann et al. 2018; Fu, Kuntz, et al. 2019; Fu, Salzman, et al. 2021), satellite scheduling, and traffic navigation under multiple constraints such as travel time, tolls, and emissions (Emmerich and Deutz 2018; Hwang and Masud 2012; Miettinen 2012; Roijers and Whiteson 2017).

A significant body of algorithmic research has addressed the challenges posed by MOS. These include adaptations of best-first search algorithms (Clímaco and Pascoal 2012; Current and Marsh 1993; Mandow and Pérez-de-la-Cruz 2010; Skriver 2000; Tarapata 2007; Ulungu and Teghem 1991), as well as specialized methods for the *Bi-Objective Search* (BOS) case, where exactly two objectives are considered (Ahmadi et al. 2021; Hernández, Yeoh, J. A. Baier, et al. 2023; Skyler, Atzmon, Felner, Salzman, Zhang, et al. 2022). Despite this progress, the computational burden of maintaining and comparing large sets of non-dominated paths (particularly due to the dominance checking operation) remains a central challenge. A recent comprehensive survey (Salzman et al. 2023) emphasizes that, while foundational methods are mature, core algorithmic questions concerning theoretical guarantees, pruning strategies, and heuristic control remain open.

In this paper, we define and characterize three mutually exclusive categories of search states: *must-expand states* (MES), *maybe-expand states* (MBES), and *never-expand states* (NES), which provide a principled taxonomy for reasoning about node expansions, pruning, and termination. We further extend this classification to individual search nodes (MEN, MBEN, NEN), illuminating critical differences from SOS, where such distinctions between states and nodes collapse due to the total ordering of nodes.

We then generalize A* (Dechter and Pearl 1985) into a unified framework, MOS-A*, that encapsulates a family of MOS algorithms. As part of MOS-A*, we introduce the notions of *Ordering Functions* (OFs) and *Tie-Breaking Policies* (TBs) to control expansion order and manage ambiguities induced by partial ordering. While OFs determine the order of node expansions (according to different costs), our analysis reveals that they do not affect completeness or the theoretical set of expanded states. Conversely, TBs (within nodes with the same costs) may significantly influence which nodes are expanded and when, even when the same set of solutions is ultimately discovered.

A major bottleneck in MOS algorithms lies in performing *dominance checks* (DCs), which verify whether a newly generated node is dominated by any existing node associated with the same state. In BOS, using lexicographic ordering (Lex) allows constant-time dominance checks (CTDC) through carefully maintained per-state thresholds (Hernández, Yeoh, J. A. Baier, et al. 2023). In this work, we introduce two non-lexicographic ordering functions, Min and Max, and introduce constant-time dominance-checking schemes for both that enable direct empirical and theoretical comparisons across orderings. Our methods maintain concise summaries of expansion progress, and we formally prove their correctness and efficiency. Notably, we demonstrate that with Min the search can terminate safely even before OPEN is empty, a phenomenon that does not occur under traditional lexicographic strategies.

These theoretical contributions are complemented by extensive empirical evaluations on commonly used benchmark problems from the DIMACS challenge suite (DIMACS 2006), covering cases with two to four objectives. Our results confirm that different ordering functions yield distinct search dynamics, and that CTDC implementations for Min and Max offer significant runtime benefits without compromising optimality.

In summary, this work establishes a general theoretical foundation for reasoning about node expansions in MOS, introduces practical algorithms that extend the benefits of CTDC beyond lexicographic orderings (e.g., Min, Max), and provides concrete guidance for selecting and implementing MOS algorithms in real-world applications.

Portions of this work have been previously published. (1) We first introduced new OFs, such as Min and Max, in SoCS-22 (Skyler, Atzmon, Felner, Salzman, Zhang, et al. 2022), in the context of a bounded-cost search for MOS. (2) Preliminary work on the theoretical parts of this paper has appeared in IJCAI-24 (Skyler, Shperberg, et al. 2024). (3) Our proposed CTDC for Min and Max was presented in AAAI-26 (Skyler, Atzmon, Felner, Salzman, Hernandez Ulloa, et al. 2026). This paper unifies these lines of work into a coherent story while significantly extending the previous papers in many respects.

Overview. Section 2 provides definitions and the relevant background for this paper. Section 3 revisits state classes from SOS and generalizes them to MOS, formalizing must-expand states and their partition. Section 4 introduces the MOS-A* framework and classifies nodes within a general, abstract algorithmic setting for MOS. Section 5 introduces ordering functions beyond lexicographic orderings (including Min and Max) and their properties. Section 6 provides a deeper treatment of NENs and differentiates between expanding nodes and extracting them from OPEN. Section 7 presents *constant-time dominance checks* (CTDC) for these orderings in BOS. Section 8 reports the empirical evaluation, and Section 9 provides a practical comparison of the OFs. Finally, Section 10 summarizes and concludes this paper.

2 Definitions and Background

In MOS, the costs of edges (and of paths) have a unique value for each objective. Thus, a common practice is to associate cost vectors with edges and paths. Here, we introduce the relevant notation. **Boldface** font (e.g., $\mathbf{p}$) indicates vectors. The notation p_i is used to denote the i 'th component of $\mathbf{p}$. Lower-case and upper-case symbols indicate elements and sets, respectively. A MOS graph G with d objectives is a tuple $\langle S, E, \mathbf{c} \rangle$, where S is the finite set of *states*, $E \subseteq S \times S$ is the finite set of edges, and $\mathbf{c} = (c_1, c_2, \dots, c_d) : E \rightarrow (\mathbb{R}_{\geq 0})^d$ is a *cost function* in the form of a d -dimensional vector (one dimension per objective) of non-negative costs. A *path* π from s_1 to s_m is a sequence of states $\{s_1, s_2, \dots, s_m\}$ such that $(s_i, s_{i+1}) \in E$ for all $i \in \{1, \dots, m-1\}$.

As stated above, we use **boldface** font to represent d -dimensional vectors in the form $\mathbf{v} = (v_1, v_2, \dots, v_d)$. For two vectors $\mathbf{u}$ and $\mathbf{v}$, $\mathbf{u} + \mathbf{v} = (u_1 + v_1, \dots, u_d + v_d)$. The cost of path $\pi = \{s_1, \dots, s_m\}$ is $\mathbf{c}(\pi) = \sum_{i=1}^{m-1} \mathbf{c}(s_i, s_{i+1})$. Given two d -dimensional vectors $\mathbf{u}$ and $\mathbf{v}$, we say that $\mathbf{u}$ *weakly dominates* $\mathbf{v}$, denoted as $\mathbf{u} \preceq \mathbf{v}$, if $u_i \leq v_i$ for every $i \in \{1, \dots, d\}$. Vector $\mathbf{u}$ *(strictly) dominates* $\mathbf{v}$, denoted as $\mathbf{u} \prec \mathbf{v}$, if $\mathbf{u} \preceq \mathbf{v}$ and $\mathbf{u} \neq \mathbf{v}$ (for at least one dimension i , $u_i < v_i$). If $u_i > v_i$ for some $i \in \{1, \dots, d\}$, then we say that $\mathbf{u}$ *non-dominates* $\mathbf{v}$ (likewise $\mathbf{v}$ is *non-dominated by* $\mathbf{u}$) and denote this by $\mathbf{u} \not\preceq \mathbf{v}$. For example, $(4, 5) \preceq (4, 5)$ (they are, in fact, equal), $(4, 5) \prec (4, 7)$ (as $5 < 7$) and $(4, 7) \not\preceq (4, 5)$. Finally, we say that two vectors $\mathbf{u}$ and $\mathbf{v}$ are *mutually non-dominated* if both $\mathbf{u} \not\preceq \mathbf{v}$ and $\mathbf{v} \not\preceq \mathbf{u}$. For example, $(4, 8)$ and $(5, 6)$ are *mutually non-dominated*. Similarly, we say that path π *dominates* (weakly dominates) path π' , denoted as $\pi \prec \pi'$ ($\pi \preceq \pi'$), if $\mathbf{c}(\pi) \prec \mathbf{c}(\pi')$ ($\mathbf{c}(\pi) \preceq \mathbf{c}(\pi')$).

Let $\mathbf{X}$ be a set of d -dimensional vectors. We say that $\mathbf{X}$ is a *mutually non-dominated set* if all pairs of vectors in $\mathbf{X}$ are mutually non-dominated (namely, no vector in $\mathbf{X}$ weakly dominates any other vector in $\mathbf{X}$).

A *search instance* is a tuple $I = \langle S, E, \mathbf{c}, \text{start}, \text{goal}, \mathbf{H} \rangle$, where $\langle S, E, \mathbf{c} \rangle$ is a MOS graph, $\text{start}, \text{goal} \in S$ are the *start* and *goal* states, and $\mathbf{H}$ is a set of heuristic functions (for each state, see below). A *solution* is a path from *start* to *goal*. A solution is *Pareto-optimal* if it is non-dominated by any other solution. A *Pareto-optimal solution set* for I , denoted by POS , is the (maximum) set of Pareto-optimal solutions. Let $\mathcal{POC} = \{\mathbf{c}(\pi) \mid \pi \in \text{POS}\}$ denote

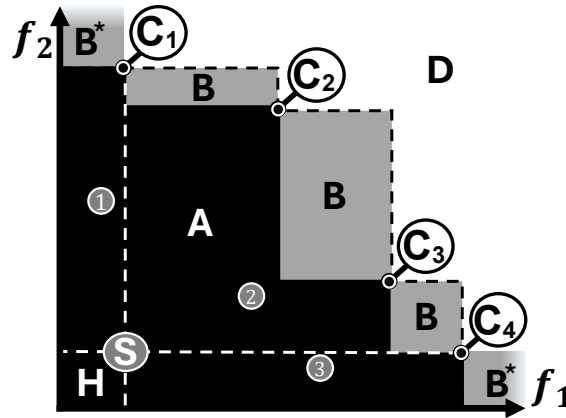


Fig. 1. Different areas in a BOS problem instance

the set of path costs in POS . By definition, any two members of POC are mutually non-dominated. However, there can be two paths with equal costs in POS . We, thus, define a *cost unique Pareto-optimal solution set*, also called the *Pareto-optimal frontier*, denoted by POF , as a subset of POS such that no two paths have equal costs. That is, there is only one path for every value in POC . In the special case of SOS (where $d = 1$), POS includes all optimal-cost (least-cost) paths while POF includes only one optimal-cost path. Figure 1 shows different cost vectors for a BOS problem (where $d = 2$). The C points denote the POC . This means that there are no solution costs in the $A \cup B$ zone. Similarly, solutions in C are mutually non-dominated and are non-dominated by any solution in Zone D (more on these zones below). The common task in MOS is to find the entire POF , i.e., one path for each possible POC value.

Single-Valued Heuristic (SVH) functions assign to each state a d -dimensional vector $\mathbf{h} = \{h_1, \dots, h_d\}$ where each value h_i estimates the cost along dimension i . Let $POC(s)$ be the path costs in POS between state s and goal. $\mathbf{h}$ is *admissible* if for each state $s \in S$, each member of $POC(s)$ is weakly dominated by $\mathbf{h}(s)$. Similarly, $\mathbf{h}$ is *consistent* iff (i) $\mathbf{h}(\text{goal}) = \mathbf{0}$ and (ii) $\mathbf{h}(s) \preceq \mathbf{c}(s, t) + \mathbf{h}(t)$ for all $(s, t) \in E$. A commonly used SVH is the *individual shortest path* heuristic, also called (*Single*) *Point* heuristic (Goldin and Salzman 2021; Hernández, Yeoh, J. A. Baier, et al. 2023; Pulido et al. 2015; Skyler, Atzmon, Felner, Salzman, Zhang, et al. 2022). That is, for $\mathbf{h}(n)$, $h_i(n)$ is the cost-minimal path from n to goal using the i -th objective only (while ignoring the other objectives). In Figure 1, S represents the value of the Point heuristic, and Zone $H \subset A$ includes all other single-valued admissible heuristics. For example, consider a problem instance with only two solution paths with costs $(1, 10)$ and $(11, 2)$. Then, the point heuristic is $\mathbf{h}(\text{start}) = (1, 2)$. Naturally, this heuristic is the best (perfect) possible heuristic for SVHs.

Multi-Valued Heuristic functions (MVH) assign to each state a set of d -dimensional vectors $\mathbf{H} = \{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ where each vector $\mathbf{h}_i$ is a heuristic vector with d dimensions. $\mathbf{H}$ is *admissible* if for each state $s \in S$, each member of $POC(s)$ is weakly dominated by at least one heuristic $\mathbf{h}_i(s) \in \mathbf{H}$. Similarly, $\mathbf{H}$ is *consistent* iff (i) $\forall_i \mathbf{h}_i(\text{goal}) = \mathbf{0}$ and (ii) $\forall_i \exists_j \mathbf{h}_i(s) \preceq \mathbf{c}(s, t) + \mathbf{h}_j(t)$ for all $(s, t) \in E$. Examples of multi-valued heuristics are the *Differential Heuristics* (Zhang et al. 2023), and other approaches introduced for multi-objective planning algorithms (Geisser et al. 2022). For example, consider a problem instance with only two solution paths with costs $(1, 10)$ and $(11, 2)$. Then, a possible admissible multi-valued heuristic $\mathbf{H}$ might be: $\mathbf{H}(\text{start}) = \{(1, 8), (9, 2)\}$. Naturally, the best possible (perfect) heuristic for multi-valued heuristics in this example would be $\mathbf{H}(\text{start}) = \{(1, 10), (11, 2)\}$. Figure 1 presents an example for MVH, where $\mathbf{H} = \{\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3\}$ such that $\mathbf{h}_1 \preceq \{C_1, C_2\}$, $\mathbf{h}_2 \preceq \{C_2, C_3\}$, and $\mathbf{h}_3 \preceq \{C_3, C_4\}$ (the different $\mathbf{h}$ -values are in numbered circles in the figure).

3 Generalizing State Classes to Multi-Objective Search

In this section, we first provide the classic classification of states in SOS (Dechter and Pearl 1985) and then generalize it to the MOS setting.

3.1 Classes of States in Single-Objective Search

Following the notation introduced by Dechter and Pearl 1985, let I_{AD} to denote problem instances that have an admissible heuristic, and let $I_{CON} \subset I_{AD}$ denote the instances that have a consistent heuristic. *Admissible algorithms* are algorithms that guarantee to return optimal solutions on all problem instances from I_{AD} . Likewise, a search algorithm is said to be DXBB (Eckerle et al. 2017) if it is deterministic and has only *black-box* access to the graph and the heuristic, i.e., it can only discover states, edges, and cost by continuously applying an *expansion* operator from either *start* or *goal*.

Admissible SOS algorithms have two main tasks: (i) finding a solution of cost C^* and (ii) proving that the returned solution is optimal. A classic claim is that any admissible DXBB unidirectional heuristic search algorithm *must expand* all states s such that there exists a path U from *start* to s for which $c(U) + h(s) < C^*$, when running on an instance from I_{CON} (Dechter and Pearl 1985)—such states are called *Must-Expand States* (MESs). To fulfill task (ii), algorithms must expand every MES s , or they might miss an optimal solution that passes through s . In addition, if $c(U) + h(s) > C^*$ for all paths U from *start* to s , then state s should *never be expanded*, as it cannot possibly lead to an optimal solution when the heuristic is admissible and, thus, does not contribute to accomplishing both tasks—such states are called *Never-Expand States* (NESs). Finally, depending on the *tie-breaking policy* (TB), some states s , where $c(U) + h(s) = C^*$, *might* be expanded in order to find an optimal solution (task (i)) and are called *Maybe-Expand States* (MBESs). Note that once a solution of cost C^* is found, the search can safely terminate without expanding all MBESs. Different TBs among nodes with $f = C^*$ may expand different subsets of MBESs until an optimal solution is found. We next generalize this theory from SOS to MOS. We distinguish between states in the underlying graph and nodes in the search tree, which are associated with partially developed paths (see below). We dedicate a separate section to each, motivated by the inherent differences between the two entities, as demonstrated in Section 4.4. It is noteworthy that a theoretical study by Mandow and Pérez-de-la-Cruz 2010 (which explores the NAMOA* algorithm) partially overlaps with our current work. Specifically, they defined the concepts of *must-expand paths* and *never-expand paths* (where paths in their notation align with nodes in our notation). As their definitions and corresponding proofs are pertinent to our discussion (Definitions 1-3 for states and Definitions 4-6 for nodes, below), we restate them here using our terminology for the sake of completeness.

3.2 Classes of States in Multi-Objective Search with Single/Multi-Valued Heuristics

The optimality proving task in SOS is generalized into two primary search tasks for MOS: (i) to identify all Pareto-optimal costs in $\mathcal{POC}$, and (ii) to verify that no additional non-dominated costs exist. To address these tasks, we generalize the classification of states from SOS to MOS, defining MESs, MBESs, and NESs for MOS. These classifications support both single-valued heuristics (SVH) and sets of admissible heuristics (MVH). In the latter case, each definition is evaluated for some individual $\mathbf{h} \in \mathbf{H}$.

DEFINITION 1 (MUST-EXPAND STATES (MESs)). A state s is a Must-Expand State (MES) iff there exists a path U from *start* to s and an admissible heuristic $\mathbf{h}$ (either single-valued $\mathbf{h}$ or some individual $\mathbf{h} \in \mathbf{H}$) such that for all $\mathbf{p} \in \mathcal{POC}$, we have $\mathbf{p} \not\leq c(U) + \mathbf{h}(s)$. That is, the estimated cost to a state s is not dominated by any known solution cost.

We further partition the MESs into two disjoint subsets: *Optimality States* and *Completeness States*, based on their relationship to the current contents of $\mathcal{POC}$.

Definition 1.1 (Optimality States). A state s is an Optimality State iff there exists a path U from start to s , an admissible heuristic $\mathbf{h}$ (SVH or $\mathbf{h} \in \mathbf{H}$), and a cost $\mathbf{p} \in \mathcal{POC}$ such that $\mathbf{c}(U) + \mathbf{h}(s) \prec \mathbf{p}$. These states contribute to proving that all elements in $\mathcal{POC}$ are non-dominated.

Optimality states are analogous to MESs in SOS (under the condition $f < C^*$). Zone A in Figure 1 corresponds to the estimated costs of such states.

Definition 1.2 (Completeness States). A state s is a Completeness State iff there exists a path U from start to s and a heuristic $\mathbf{h}$ (either single-valued $\mathbf{h}$ or some individual $\mathbf{h} \in \mathbf{H}$) such that for all $\mathbf{p} \in \mathcal{POC}$: $\mathbf{c}(U) + \mathbf{h}(s) \not\prec \mathbf{p}$ and $\mathbf{p} \not\prec \mathbf{c}(U) + \mathbf{h}(s)$. That is, the estimated cost is mutually non-dominated with respect to all known Pareto-optimal costs.

Completeness states ensure that no further solutions can be added to $\mathcal{POC}$. They have no analog in SOS. Their corresponding cost region is shown in Zone B of Figure 1.

Definition 2 (NEVER-EXPAND STATES (NESS)). A state s is a Never-Expand State (NES) iff for every path U from start to s and every admissible heuristic $\mathbf{h}$ (or every $\mathbf{h} \in \mathbf{H}$), there exists a cost $\mathbf{p} \in \mathcal{POC}$ such that $\mathbf{p} \prec \mathbf{c}(U) + \mathbf{h}(s)$. These states are guaranteed to be dominated by already discovered solutions.

NESSs are analogous to states pruned in SOS due to $f > C^*$. They correspond to Zone D in Figure 1.

Definition 3 (MAYBE-EXPAND STATES (MBES)). A state s is a Maybe-Expand State (MBES) iff s is not a MES, and there exists a path U from start to s , an admissible heuristic $\mathbf{h}$ (or $\mathbf{h} \in \mathbf{H}$), and a cost $\mathbf{p} \in \mathcal{POC}$ such that $\mathbf{c}(U) + \mathbf{h}(s) = \mathbf{p}$.

MBESs represent states whose estimated costs lie exactly on the Pareto front. Depending on the tie-breaking, such states may or may not be expanded. They correspond to Zones C_1 - C_4 in Figure 1, and are analogous to states with $f = C^*$ in SOS.

3.3 Theoretical Analysis

The definitions above were formulated to apply both to the *single-vector* (point) heuristic case (SVH) and to *multi-vector* (multi-valued) heuristics (MVH). In the remainder of this paper, we mainly focus on the more commonly used single-valued heuristics for clarity and consistency. Nevertheless, all theoretical claims presented herein generalize naturally to the multi-valued case.

We next formally establish that every Must-Expand State (MES) must be expanded to ensure optimality (i.e., to satisfy task (ii)), and that Never-Expand States (NESSs) are irrelevant to both discovering optimal solutions (task (i)) and proving their optimality (task (ii)).

Lemma 1. Given a problem instance $I \in I_{CON}$, any admissible MOS Algorithm A must expand all MESs (Def. 1).

PROOF. Let $\mathcal{POC}_I$ be the set of path costs of a POF of instance I . To prove the lemma, we need to show that every state s for which there exists a Pareto-optimal path U from start to s and there exists $\mathbf{p} \in \mathcal{POC}_I$ and $\mathbf{h} \in \mathbf{H}$ such that $\mathbf{p} \not\prec \mathbf{c}(U) + \mathbf{h}(s)$, then s must be expanded by Algorithm A. We prove the contrapositive. Suppose that (i) $I_1 = \langle G_1 = \langle V, E, \mathbf{c}_1 \rangle, \text{start}, \text{goal}, \mathbf{H} \rangle$ is a problem instance in I_{CON} whose POF has a set of costs $\mathcal{POC}_{I_1}$, (ii) U is a Pareto-optimal path from start to s in I_1 and $\mathbf{h} \in \mathbf{H}$ such that $C^* \not\prec \mathbf{c}_1(U) + \mathbf{h}(s)$ for all $C^* \in \mathcal{POC}_{I_1}$, (i.e., $\mathbf{c}_1(U) + \mathbf{h}(s)$ is in Zone $A \cup B$), and (iii) A is an admissible DXBB algorithm that returns a POF for I_1 , denoted $A(I_1)$ (with cost vectors $\mathcal{POC}_{I_1}$), without expanding s . Then, a new instance $I_2 \in I_{AD}$ can be constructed, whose POF has a set of cost vectors $\mathcal{POC}_{I_2}$, such that (i) there exists $C \in \mathcal{POC}_{I_2}$ for which $C \not\prec C'$ for all $C' \in \mathcal{POC}_{I_1}$, and (ii) A also returns $A(I_1)$ (that contains no solution of cost C), thereby showing that A is not admissible on I_2 . $I_2 = \langle G_2 = \langle V, E \cup \{e = \langle s, \text{goal} \rangle\}, \mathbf{c}_2 \rangle, \text{start}, \text{goal}, \mathbf{H} \rangle$. G_2 is copied from G_1 , with the following changes: (i) an edge from s to goal is added (if it did not exist before) and (ii) the cost function $\mathbf{c}_2$ is identical to $\mathbf{c}_1$ with a single

modification, $c_2(e) = \mathbf{h}(s)$ ($\mathbf{h}(s) \in \mathbf{H}(s)$). Edge e generates a solution path $U' = [start, \dots, s, goal]$ whose total cost is $c_2(U') = c_1(U) + \mathbf{h}(s)$, which is non-dominated by any cost in $\mathcal{POC}_{I_1}$ (by assumption (ii)). Edge e is thus an essential part of any *POF* of I_2 . Since A is DXBB, it will behave on I_2 similarly to its behavior on I_1 . In particular, A will not expand s , will not discover edge e , and will incorrectly return $A(I_1)$ as the *POF* for I_2 . Hence, if $I_2 \in I_{AD}$, A is not admissible.

To complete the proof, we now show that $I_2 \in I_{AD}$, i.e., that $\mathbf{h}$ is admissible on G_2 (even after edge e was added). Let x be an arbitrary state in G_2 and let W be any acyclic path in G_2 from x to *goal*. We need to prove that $\mathbf{h}(x) \preceq c_2(W)$. If W does not contain the new edge e , the claim trivially follows from the admissibility of $\mathbf{h}$ on G_1 . Now, suppose W does contain e . In other words, $W = \{X, e\}$ for some path X from x to s (i.e., e is the last edge of W). Now, since X is a path from x to s in G_1 and $\mathbf{h}$ is consistent on G_1 we have:

$$\mathbf{h}(x) \preceq c_1(X) + \mathbf{h}(s)$$

By definition, $c_2(e) = \mathbf{h}(s)$ and $c_1(X) = c_2(X)$, thus:

$$= c_2(X) + c_2(e) = c_2(W)$$

Therefore, $\mathbf{h}$ is admissible on G_2 , i.e., $I_2 \in I_{AD}$. $\square$

We next show that NESs should never be expanded by admissible search algorithms.

LEMMA 2. *Given an instance $I \in I_{AD}$, every NES (Def. 2) cannot lead to any path with cost $\mathbf{p} \in \mathcal{POC}_I$.*

PROOF. Assume by contradiction that (i) there exists a problem instance I such that the costs of all paths from *start* to some state s in I are dominated by one of the values in $\mathcal{POC}_I$ and that (ii) s must be expanded by any admissible MOS Algorithm A in order to find a *POF* of I (denoted as $A(I)$). Let $W = XY$ be an acyclic path from *start* to *goal* such that X is a path from *start* to s and Y is a path from s to *goal*. Due to the admissibility of I , we know that $\mathbf{h}(s) \preceq c(Y)$ for all paths Y from s to *goal* and $\mathbf{h}(s) \in \mathbf{H}(s)$. Thus, we get:

$$c(W) = c(X) + c(Y) \succeq c(X) + \mathbf{h}(s)$$

and, due to the assumption:

$$\succeq \mathbf{p} \text{ for some } \mathbf{p} \in \mathcal{POC}_I$$

Thus, all paths W that pass through s are not a *POF* solution, and s should not be expanded. $\square$

4 The MOS-A* Framework and Classification of Nodes

Until now, we have discussed the classification of *states*. However, heuristic search algorithms are typically defined over search *nodes*, each representing a specific path in the graph. In SOS with a consistent heuristic, the f -value of nodes is monotonically non-decreasing along paths. When *duplicate detection* is employed, and multiple paths to a state s exist, A^* associates s with a single node $n(s)$ that corresponds to the cost-minimal path found so far. All other paths to s are pruned. As a result, selecting a node $n(s)$ with minimal f -value in OPEN ensures that $n(s)$ corresponds to an optimal path from *start* to s (Dechter and Pearl 1985).

These properties enable seamless translation from state-based to node-based classification in SOS. Specifically, given a heuristic function h , the A^* algorithm (i) must expand all nodes n such that $f(n) < C^*$, (ii) should never expand nodes with $f(n) > C^*$, and (iii) may expand some nodes with $f(n) = C^*$. This behavior underlies the well-known result that A^* is *optimally efficient* under these assumptions.

To establish an analogous node-level framework in MOS and to enable a similar transition from states to nodes, we must first generalize A^* to handle vector-valued costs and partial orders. We achieve this by introducing the MOS-A* framework, a principled extension of A^* to the multi-objective setting. MOS-A* retains the general best-first search structure of A^* , but replaces scalar comparisons with Pareto dominance checks over cost vectors. This generalization enables a unified treatment of search nodes in MOS, where each node represents a unique cost vector along a path. Importantly, many existing MOS and BOS algorithms can be viewed as specific instances

Algorithm 1: MOS-A*

```

Input :  $G = \langle S, E, c \rangle, start, goal, H, OF, TB$ 
1  $n_{root} \leftarrow$  new node at  $start$  with  $g(n_{root}) = 0$  and  $parent(n_{root}) = null$ 
2 initialize OPEN with  $OF$  and  $TB$ 
3 for  $h = h(start) \in H(start)$  do
4    $h(n_{root}) \leftarrow h$ 
5   OPEN.insert( $n_{root}$ )
6  $POF \leftarrow \emptyset$ 
7 while OPEN  $\neq \emptyset$  do
8    $n \leftarrow$  OPEN.pop() // using  $OF$  and  $TB$ 
9   if dominance_check( $n$ ) then continue
10  if  $s(n) = goal$  then
11    add  $n$  to  $POF$ 
12    continue
13  for  $s' \in S$  s.t.  $(s(n), s') \in E$  do
14    for  $h = h(s') \in H(s')$  do
15       $n' \leftarrow$  new node at  $s'$  with  $g(n') = g(n) + c((s, s'))$ ,  $h(n') = h$ , and  $parent(n') = n$ 
16      if dominance_check( $n'$ ) then continue
17      OPEN.insert( $n'$ )
18 return  $POF$ 

```

of MOS-A* which differ in their tie-breaking rules, ordering functions, or data structures. In what follows, we formally define MOS-A* and demonstrate how its node-level behavior aligns with the theoretical classifications introduced in the previous section.

4.1 The Multi-Objective Search A* Framework (MOS-A*)

A* stores all frontier nodes in a priority queue called OPEN and all previously expanded nodes in a set called CLOSED. All nodes with the smallest f -value are said to be at the *front* of OPEN and A* must choose for expansion a node from *front*. Since several nodes may have the same minimal f -value, a tie-breaking policy (TB) is used to select a specific node.

We now describe a generalization of A* to MOS, which we call MOS-A*. MOS-A* uses the same expansion mechanism just described that includes OPEN and CLOSED. In MOS-A*, OPEN nodes are associated with $\mathbf{f}$ -values, which are d -dimensional vectors of values. Thus, in MOS-A*, the *front* of OPEN includes all nodes with *non-dominated* (but may be equal, $\not\prec$ relation) $\mathbf{f}$ -vectors. *front* can include multiple cost vectors (e.g., $(1, 2)$ and $(2, 1)$), each of which can be used to choose nodes for expansion. Thus, to choose a node from the *front* of OPEN, we have two phases. The first phase is unique to MOS (and does not exist in SOS) and is called an *Ordering Function* (OF) (Skyler, Atzmon, Felner, Salzman, Zhang, et al. 2022). An OF defines a total order among the different $\mathbf{f}$ -values. That is, even mutually non-dominated vectors are now ordered by the given OF. The cluster of nodes with the minimal order (according to the OF) is first chosen. A common family of OFs is the *lexicographic order*, which has been used by many algorithms because it offers significant advantages in CPU time overhead and data structure (Hernández, Yeoh, J. A. Baier, et al. 2023; Mandow and De La Cruz 2005; Pulido et al. 2015;

Stewart and White III 1991). We discuss and compare this and other OFs below. The second phase uses TB within the chosen cluster (if it has more than one node) to choose the final node to expand.

The pseudocode for the MOS-A* framework is shown in Algorithm 1. It accepts a MOS instance, an OF, and a TB. MOS-A* maintains an OPEN list of generated but not-yet-expanded nodes. OPEN is initialized in Line 2 with an OF and $\mathbf{f}$ function. Each node n contains a state $s(n)$, and a $\mathbf{g}$ -value vector $\mathbf{g}(n)$. $\mathbf{f}(n)$ is then calculated as $\mathbf{f}(n) = \mathbf{g}(n) + \mathbf{h}(s(n))$.¹ The *front* of OPEN includes all nodes whose $\mathbf{f}(n)$ -values are non-dominated (but may be equal, $\not\prec$ relation). In each iteration, MOS-A* extracts a node n with the “best” $\mathbf{f}$ -value from the *front* of OPEN, as determined by OF and TB (Line 8). MOS-A* then performs the common expansion cycle and either adds a node to *POF* if $s(n) = \text{goal}$ (Lines 10-12) or expands it (Lines 13-17). When expanding a node n , MOS-A* generates a child node for each successor s' of $s(n)$. When OPEN becomes empty, it terminates and returns the *POF*.

4.2 Dominance Checks in MOS-A*

In MOS, state s may be included in multiple nodes, corresponding to (cost-unique) non-dominated paths from *start* to s . Here, we perform *dominance-checks* (DCs) to prune weakly-dominated paths. MOS-A* performs DCs for node n to determine whether n or n 's child nodes have the potential to be included in *POF*. Otherwise, n is pruned if it fails the check. This is similar to the duplicate-detection mechanism optimally employed by A*.

In MOS-A*, *dominance checks* can be performed in two locations: after a node was chosen for expansion (Line 9) or after a node was generated (Line 16) and before it is inserted into OPEN. There are two types of checks that need to be performed on node n .

The first is called *Path Dominance Check* (PDC). Here, we compare different occurrences of $s(n)$ and prune those occurrences whose path costs are weakly dominated by the costs of other occurrences. After this, only non-dominated nodes (with $s(n)$ as their state) are left in OPEN and CLOSED. This is a generalization of *duplicate detection* in SOS, where we only keep one node for each state with minimal path cost. Similar to SOS, this step is optional. One may choose not to implement it and risk a large increase in nodes added to OPEN as many of them are redundant. This directly translates into CPU time and memory increase. The tradeoff is that PDC requires hard-tailored data structures.

The second type is called *Solution Dominance Check* (SDC). Here, we prune node n if $\mathbf{f}(n)$ is dominated by a previously found solution. This dominance check is mandatory. Without it, the algorithm can run forever, producing solutions indefinitely. SDC is not performed in SOS because the search usually halts after finding the first solution. But, it is equivalent to pruning nodes with $f > C^*$ in SOS. This will happen in practice if, for example, the search task is to continue searching after the first optimal solution is found and return all optimal solutions.

Numerous algorithms adopt the MOS-A* framework, including MOA* (Stewart and White III 1991), NAMOA* (Mandow and De La Cruz 2005), NAMOA*-dr (Pulido et al. 2015), and BOA* (Hernández, Yeoh, J. Baier, et al. 2023; Hernández, Yeoh, J. A. Baier, et al. 2023). While all of these use lexicographic OF, they differ primarily in the exact location where DCs are performed in their pseudocode and in the data structures used to manage OPEN.

In both MOA* and NAMOA*, every occurrence of a state is preserved along with its corresponding $\mathbf{g}$ -values. Upon re-encountering a previously visited state, the current node's $\mathbf{g}$ -value is compared against all earlier instances of that state to determine dominance. NAMOA*-dr, a dimensionality-reduction variant of NAMOA*, leverages the fact that nodes are extracted from OPEN in lexicographic order, thereby obviating the need to compare the first objective. Consequently, only partial vectors (excluding the first dimension) are stored for each state in NAMOA*-dr's data structures.

¹In the case of MVH, the common practice is to generate a separate node for each $\mathbf{h}(s(n)) \in \mathbf{H}(s(n))$. This adds CPU and memory overhead and complicates the usage of MVHs.

Within the BOS context, eliminating the first dimension yields a single relevant cost value, specifically g_2 , the cost in the second dimension. Under lexicographic ordering, if nodes are extracted from OPEN by increasing g_1 , the corresponding g_2 values must decrease monotonically to avoid dominance. BOA* exploits this property to enable constant-time dominance check (CTDC) by maintaining, for each state, the minimum g_2 value of any previously expanded node. CTDC has a significant effect on the constant time per node expansion, and several orders of magnitude speedup in CPU time may be obtained. In Section 7, we further delve into CTDC and introduce a novel method that performs constant time DC for Min and Max extending the approach from Lex to other OFs for the bi-objective case.

Note that MOS-A* can be generalized to any best-first search algorithm by replacing f with the corresponding function. For example, to adapt MOS-A* to Greedy Best-First Search (Doran and Michie 1966), which expands nodes with minimal h -value, the only modification required to MOS-A* is to sort OPEN with respect to h (Line 2).

We now show how the classification of states for MOS can be adapted to the classification of nodes when considering this family of MOS-A* algorithms.

4.3 Node Classification in MOS-A*

In MOS-A*, any state s is represented by a set of nodes $N(s)$, where each node $n \in N(s)$ corresponds to a non-dominated path from *start* to s . Similarly to SOS, it was shown that, given a consistent set of heuristics, the f -value of nodes is monotonically non-decreasing along any path (Stewart and White III 1991). Thus, only expanding a node $n \in N(s)$ for which $f(n)$ is non-dominated by any other f -values in OPEN ensures that n represents a Pareto-optimal path from *start* to s . Consequently, the different classifications of states can be adapted to nodes, as we now show for MOS in general, but only illustrate for the special case of BOS in Figure 1.²

DEFINITION 4 (MUST-EXPAND NODES (MENs)). *Given a problem instance $I \in I_{CON}$, a node n is a MEN if $f(n)$ is non-dominated by any cost in any POF of instance I , $\mathcal{POC}_I$, i.e., $\forall p \in \mathcal{POC}_I$ it holds that $p \not\prec f(n)$.*

Here too, the definition is split into two subcategories:

Definition 4.1 (Optimality Nodes). *Optimality nodes n dominate one solution (or more) from any POF, i.e., $\exists p \in \mathcal{POC}$ such that $f(n) \prec p$.*

Optimality nodes (Zone A in Figure 1) are analogous to the MENs in SOS (with $f(n) < C^*$) and must be expanded to ensure the optimality of solutions in any POF.

Definition 4.2 (Completeness Nodes). *Completeness nodes n are mutually non-dominated by any solution in any POF, i.e., $\forall p \in \mathcal{POC}$ it holds that $f(n) \not\prec p$ and $p \not\prec f(n)$.*

Completeness nodes (Zone B in Figure 1) have no analogy in SOS and have to be expanded to ensure there are no more solutions in any POF (i.e., that the POF is complete).

In BOS, the minimal cost on the first objective (c_1 of C_1 , the top-left solution in Figure 1) has the maximal cost on the second objective (c_2), and the minimal cost on the first objective (c_1 of C_4 , the bottom-right solution in Figure 1) has the maximal cost on the second objective (c_2). We denote the top-left and bottom-right solutions as *extreme solutions*. Note that in BOS, area B^* can be pruned if the *extreme solutions* are known, which is common when using *Single-Point Heuristic* (reminder, a single vector of heuristics, one for each dimension) (Skyler, Atzmon, Felner, Salzman, Zhang, et al. 2022).

DEFINITION 5 (NEVER-EXPAND NODES (NENs)). *NENs n are dominated by at least one path in any POF, i.e., $\exists p \in \mathcal{POC}$, such that $p \prec f(n)$.*

²For simplicity, we only consider SVH. But, definitions can be easily generalized to MVH as follows. When we refer to $f(n)$, then for MVH we should say that $\exists h \in H$ and we refer to $g(n) + h(n)$.

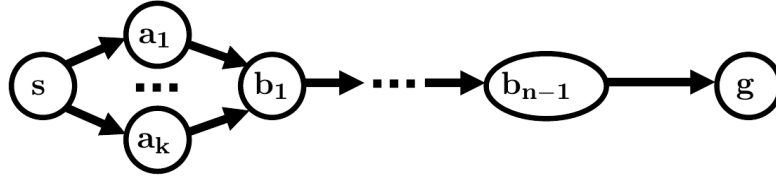


Fig. 2. Information propagation back from the goal.

NENs are analogous to nodes with $f(n) > C^*$ in SOS and belong to Zone D , which also includes the (dashed) border with Zone B in Figure 1.

DEFINITION 6 (MAYBE-EXPAND NODES (MBENs)). *MBENs are nodes n that are neither MENs nor NENs, i.e., $\exists p \in POC$ such that $f(n) = p$.*

All nodes in Zones C in Figure 1 are MBENs, which are analogous to nodes with $f(n) = C^*$ in SOS.

The proofs of Lemmas 1 (for MES) and 2 (for NES) can be directly adapted for nodes when using MOS-A* algorithms and the above definitions.

LEMMA 3. *Any MOS-A* algorithm expands all MENs and does not expand NENs.*

PROOF. Since MOS-A* algorithms only expand nodes with non-dominated $\mathbf{f}$ -values and only prune dominated nodes, they are guaranteed to expand all MENs. We thus need to prove that MOS-A* algorithms do not expand NENs. Assume by contradiction that a MOS-A* Algorithm A expands a NEN n . Since n is a NEN, there exists a Pareto-optimal solution π such that $\pi \prec \mathbf{f}(n)$. Since MOS-A* algorithms only expand nodes with non-dominated $\mathbf{f}$ -values, π would be discovered before n is expanded. Thus, n will be pruned during the solution dominance check in Line 9 of Algorithm 1, in contradiction to n being expanded by A . $\square$

4.4 State-Based vs. Node-Based Perspectives

In the context of MOS, the definitions of Must-Expand (ME), Maybe-Expand (MBE), and Never-Expand (NE) differ between nodes and states. With a consistent heuristic, the number of states expanded is bounded by the graph's size. By contrast, the number of nodes representing cost-unique paths can be exponential in the number of states. Generally, the size of the Pareto-optimal frontier can be exponential in the number of states and thus counting states expanded may significantly *underestimate* the time and memory required by the algorithm.³ By contrast, considering node expansions can be too restrictive, potentially *overestimating* the resources needed by the search as we now demonstrate using Figure 2. There are k states, $a_1, \dots, a_k$, connected to s (start). Each of these k states is further linked to a series of states labeled $b_1, \dots, b_n = g$, forming k Pareto-optimal paths: $s, a_i, b_1, \dots, b_n, g$, for $1 \leq i \leq k$. In currently existing MOS-A* algorithms, the search is carried out independently across distinct search nodes (paths). Thus, each node a_i ($1 \leq i \leq k$) will entail a distinct expansion of state $b_1, \dots, g$ (forming a distinct node for each of them). In this context, the definitions of MENs and NENs hold significance.

However, under the DXBB assumption, an algorithm A may first expand $s, a_1, b_1, \dots, g$ and discover the first Pareto-optimal path. A can then propagate information, e.g., the path and its costs backward from g to b_1 . Since the expansion function is defined on states (and not paths), when a_2 is expanded, the path from b_1 to the goal is already known. As a result, $b_1, \dots, b_n$ need not be expanded again. This also applies to the expansion of $a_3, \dots, a_k$.

³In the special case of SOS, there is only one node $n(s)$ for each state s which contains the shortest path to s found thus far. $n(s)$ is expanded when the shortest path to s is known.

In these scenarios, where certain states need not be re-expanded, MENs and NENs may overestimate the number of expanded nodes, whereas MESs and NESs remain valid.

We note that algorithms with such backpropagation and information sharing do not yet exist and are the focus of active research. We thus next examine the influence of TBs and OFs in the context of MEN, MBEN, and NEN.

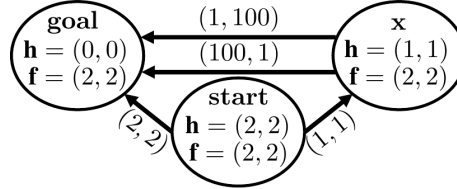


Fig. 3. Tie-breaking example in MOS-BF*

4.5 Role of Tie-Breaking Policies in MOS-A*

Tie-breaking policies (TBs) determine which node to expand, among all nodes with the same f -value in OPEN. Consequently, different TBs can result in different nodes being expanded, even when the same OF is used. An example of the TBs' effect on the search is shown in Figure 3. There are three states, *start*, *x*, and *goal*. Any MOS-A* must expand *start* first (with any OF), generating nodes *x* and *goal* (both have the same $f = (2, 2)$). Thus, the order in which they are expanded is solely determined by the TB. Consider the following two TBs: $p_{h\uparrow}$, which expand nodes in increasing order of their sum of h -values, and $p_{h\downarrow}$, which expand nodes in decreasing order of their sum of h -values. Using $p_{h\uparrow}$, *goal* is expanded first (after *start*), finding a solution of cost $(2, 2)$. In this case, the algorithm can terminate without expanding the node of *x*, as a solution that weakly dominates this node has been found (using the solution dominance check in Line 7 of Algorithm 1). By contrast, $p_{h\downarrow}$ expands *x* and only then expands *goal*, resulting in three node expansions instead of two.

4.6 Role of Ordering Functions in MOS-A*

Different TBs can result in different node expansions. We now show that using different ordering functions (OFs) with the same fixed TB, results in the *same* set of node expansions.

LEMMA 4. *Let O_1 and O_2 be two OFs and let tb be a deterministic TB. A MOS-A* Algorithm A that expands nodes according to O_1 and tb will expand the exact set of nodes on problem instance $I \in I_{CON}$ when using O_2 and tb .*

PROOF. We prove the contrapositive. Let n be the first node expanded by A using O_1 and not expanded using O_2 . Since A is admissible, we know that n is not a MEN. Otherwise, A must expand n even when using O_2 . In addition, n is not a NEN (Lemma 3). Thus, n is a MBEN and $f(n) \in \mathcal{POC}$. A could avoid expanding n only by finding a solution of cost $f(n)$ that does not pass through n . That solution must be discovered by O_2 by expanding some other node n' that is not expanded by O_1 , with $f(n') \preceq f(n)$ (due to admissibility). However, since $f(n') \prec f(n)$, n' would have been expanded before n when using O_1 and n would have been pruned due to the solution dominance check, contradicting the fact that n was expanded using O_1 . Thus, $f(n') = f(n)$. Since n is the first node that is not expanded by A using O_2 , both n and n' must have been in OPEN before either is expanded. The same deterministic tie-breaking tb is used in both runs. Therefore, the order by which n and n' are expanded is identical in the two runs, in contradiction to n' being expanded before n when using O_1 . $\square$

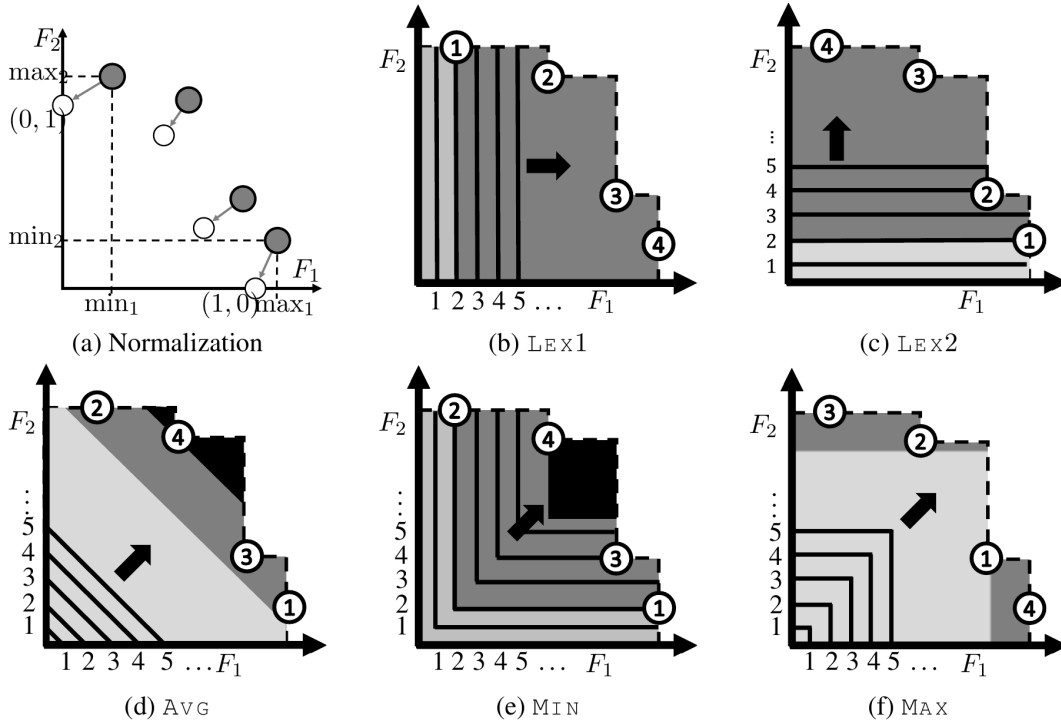


Fig. 4. (a) min-max Normalization, the Grey circles are the costs before normalization, and the White circles are the costs after the normalization. The extreme solutions are $(0, 1)$ and $(1, 0)$ respectively. (b)-(f) The progress of different ordering functions. The three phases are colored differently.

5 Ordering Functions

While ordering functions (OFs) do not change the final set of nodes expanded during the search, they can affect the *order* by which these nodes are expanded and thus the order and rate by which solutions are discovered. This may be crucial in *anytime scenarios*, where one needs many solutions as fast as possible. We now describe several OFs and analyze their effect on the search. Figure 4 illustrates the progress of the different OFs on the search in the bi-objective search (BOS) setting. The search is divided into three phases: (1) The First phase – nodes that are expanded before the first solution is found (the light gray area); (2) The Mid phase – Nodes that are expanded after finding the first solution but before finding the last solution (the dark gray area); and (3) The Last phase – Nodes that were expanded after the last solution was found (the black area). These nodes are expanded to verify that no more solutions exist. The arrows in the figures indicate the order in which nodes are expanded.

5.1 Normalizing the Range of Values

Frequently, the objectives are measured on different scales, e.g., time in seconds and distance in miles. Thus, to compare them meaningfully, we normalize them to be on the same $[0, 1]$ scale. [Skyler, Atzmon, Felner, Salzmann, Zhang, et al. 2022](#) introduced a normalization procedure for BOS, which can also be applied to MOS as we describe next. Let $\min_i = \min_{c \in \mathcal{POC}} c_i$ and $\max_i = \max_{c \in \mathcal{POC}} c_i$. All the values between $\min_i$ and $\max_i$ are then linearly

projected into the $[0, 1]$ range. This is achieved by the following projection function applied to any value x_i :

$$\bar{x}_i = \frac{x_i - \min_i}{\max_i - \min_i} \quad (1)$$

Anchoring the normalization to the minimal costs at the start state (and the maximal costs observed in the *POF*) ensures a total order of pairs of values. This normalization is illustrated for BOS in Figure 4a, where the grey and white circles are the solutions before and after the normalization.

$\min_i$ and $\max_i$ values for BOS are calculated by running a SOS algorithm for objective i until *all* optimal solutions are found. $\min_i$ is set to the cost of the optimal solution. $\max_j$ (for the other dimension $i \neq j$) is set to be the minimal value of objective j in all optimal solution costs that were found for objective i . This procedure can only be done for BOS (with $d = 2$). In MOS with $d > 2$, this is not possible. For example, consider an instance where $\mathcal{POC} = \{(2, 4, 10), (3, 3, 100), (4, 2, 10)\}$. Applying this method (i.e., running A* for each objective) would only find $\{(2, 4, 10), (4, 2, 10)\}$. Thus, the maximal values vector will be $(4, 4, 10)$ instead of $(4, 4, 100)$. So, in practice, one can take any upper (lower) bound on the maximal (minimal) solution for $\max_i$ ($\min_i$). Then, the normalized value $\bar{x}$ can be calculated based on these bounds using Equation 1. We next describe the different OFs.

5.2 Lexicographical Ordering (Lex)

Let $\mathbf{p} = (p_1, \dots, p_d)$ be a permutation of $[1, \dots, d]$. The total order of the $\mathbf{f}$ -values is the *lexicographical order* induced by $\mathbf{p}$ (i.e., first compare f_{p_1} ; in the case of a tie, compare f_{p_2} ; and so on). Given a MOS problem with d objectives, there are $d!$ such permutations, each associated with its own Lex function. Figures 4b and 4c show Lex1 and Lex2 (resp.) in BOS. Lex1 finds the top-left solution first while Lex2 finds the bottom right solution first. As mentioned above (Section 4.2), Lex was used by many existing instantiations of MOS-A* due to the advantage of dimensionality reduction (discussed deeply below in Section 7).

5.3 (Weighted) Average Ordering (Avg)

Assume two nodes n and m with $\mathbf{f}$ -values $\mathbf{f}(n) = [f_{n_1}, f_{n_2}, \dots, f_{n_d}]$ and $\mathbf{f}(m) = [f_{m_1}, f_{m_2}, \dots, f_{m_d}]$, respectively. Let $\mathbf{w}$ be a weight vector. Avg is defined as follows:

$$\text{Avg}(n, m) = \arg \min \left(\sum_{i=1}^d w_i \cdot f_{n_i}, \sum_{i=1}^d w_i \cdot f_{m_i} \right)$$

If $\mathbf{w} = [1, \dots, 1]$, then it is considered to be the *sum OF*. A similar order would result from $\mathbf{w} = [1/d, \dots, 1/d]$, which is the mathematical *average* of the different f_i -values. Figure 4d illustrates Avg. The $\mathbf{f}$ -values of expanded nodes form a diagonal line that proceeds along the direction of the black arrow. Assuming the $\mathbf{f}$ -values are normalized, the slope of this diagonal line is determined by considering the exact values in $\mathbf{w}$.

5.4 Maximum Ordering (Max)

Max ranks nodes based on the maximum of their normalized objective values. For two nodes n and m , we say that

$$n \leq_{\max} m \text{ if } \max(f_1(n), f_2(n)) \leq \max(f_1(m), f_2(m))$$

Practically, for each node, the objectives are first normalized (see Section 5.1) and then placed back in the vector sorted in *descending* order. Lexicographic comparison is subsequently applied to the sorted vectors of the different nodes: a node n is ordered before a node m if the largest normalized objective of n is smaller than that of m ; in case of a tie, the next largest objective is considered, and so on. For example, in BOS, if we have two nodes with normalized values $x = 4, 8$ and $y = 3, 9$, then x will be ordered before y because its largest value 8 is smaller than the largest value of y , which is 9.

This ordering naturally prioritizes nodes with lower maximum cost among the objectives. As a result, Max tends to promote solutions that minimize the worst-performing criterion, thus favoring Pareto-optimal points with relatively balanced objective values. These solutions often lie near the central region of the Pareto front.

Despite its focus on balanced trade-offs, Max has a notable drawback in its discovery order: it significantly delays the identification of the first non-dominated solution. As demonstrated in our empirical analysis (Table 1), the algorithm often expends a substantial fraction of its total search effort—frequently exceeding 50% of all node expansions—before reaching a single solution. This delayed onset is visually depicted in Figure 4f, which illustrates the behavior of node expansion under Max ordering.

5.5 Minimum Ordering (Min)

Here we compare the minimum among the objectives. For two nodes n and m , we say that

$$n \leq_{\min} m \text{ if } \min(f_1(n), f_2(n)) \leq \min(f_1(m), f_2(m))$$

Practically, Min sorts the normalized objective values in *ascending* order. Then, lexicographic comparison is applied to the sorted vectors of the different nodes: a node n is ordered before a node m if the smallest normalized objective of n is smaller than that of m ; in case of a tie, the next-smallest objective is considered, and so on. In our BOS example from above, y will be ordered before x because its smallest value 3 is smaller than the large value of x , which is 4.

This leads the search to favor nodes that are extreme with respect to at least one objective—alternating emphasis between objectives as the front is explored. As a result, the search tends to cover the Pareto front’s boundaries early, eventually converging on the middle region. Figure 4e illustrates the Min ordering behavior.

Notably, this behavior resembles that of BOBA* (Ahmadi et al. 2021) (Bi-Objective Bi-Directional A*), which also explores the Pareto front by alternating objectives. However, the resemblance is superficial. BOBA* performs a backward search from *goal* using the second objective, while simultaneously executing a forward search from *start* using the first. The two meet in the middle through a bi-directional strategy. In contrast, Min operates under the unified MOS-A* framework and performs a single *unidirectional* search, without relying on explicit backward exploration.

6 Deeper Treatment of NENs

We next further deepen into MOS-A* and show, in MOS/BOS, that even though NEMs are indeed never expanded, yet they *must* be extracted and cleared from OPEN. This is unlike SOS, where such nodes can remain indefinitely in OPEN.

6.1 Life Cycle of a Node

In best-first search algorithms (in SOS, BOS, and MOS), it is common to say that when nodes are *expanded*, their successors are *generated*. However, there are additional steps that occur during the life cycle of a node n , as follows.

Step 1 (generation): An operator is applied on the expanded parent node p and a new node n is being *generated* (by copying p and performing the operator on it).

Step 2 (check): Node n is *checked* and might be pruned by the optional *Duplicate Detection* in SOS, by the PDC in BOS/MOS, or by the mandatory SDC in BOS/MOS. If the node is dominated, it is pruned. This step can be done eagerly after a node is generated (Step 2, Line 12 in Alg. 1) and/or lazily after the node is extracted from OPEN (Step 5, Line 9 in Alg. 1).

Step 3 (insertion): If node n was not pruned, it is being *inserted* to and maintained in OPEN for further treatment.

Step 4 (extraction): Node n is chosen to be *extracted* from OPEN based on its f -value (and on the OF and tie-breaking rule that are used in case of BOS/MOS).

Step 5 (second check): Similar to Step 2, the extracted node n is again *checked* and might be pruned. (Line 9 in Alg. 1).

Step 6 (goal test): Node n undergoes a *goal test*. If n is a goal, then, in SOS, the algorithm halts, while, in BOS/MOS, n is added to *POF*.

Step 7 (expansion): If the extracted node n is not a goal, it is *expanded*. That is, we apply all relevant operators in order to generate each of its successors (Step 1).

6.2 Extracting NENs from OPEN

As we showed above, the total number of *expanded* nodes (MEN+MBEN) is fixed across all possible OFs. The different OFs expand exactly the same set of nodes, albeit in a different order. Thus, different OFs may reach solutions in a different order and at a different pace. For example, Min finds the entire *POF* sooner in the search than Lex. Nevertheless, we want to elaborate on some attributes of NENs (Zone D). While all nodes in Zone D are never expanded, they are not *surplus* in the sense that those nodes that were inserted to OPEN must be cleared from OPEN before termination of the algorithm. This section examines this issue.

As mentioned above, in SOS, NENs (with $f > C^*$) might be inserted to OPEN, but they will never be touched again, i.e., they will never be chosen for extraction. The reason is that they can be potentially chosen for extraction/expansion only if their f -value is the best in OPEN. But this will not happen because the search halts when a goal node with $f = C^*$ is (or all goal nodes are) extracted. Thus, NENs are designated as *surplus* (Goldenberg et al. 2014) as they stay intact in OPEN. Furthermore, NENs are inserted into OPEN because, at the time of their generation, the value of C^* is unknown.

By contrast, MOS involves identifying all Pareto-optimal solution costs (*POC*) rather than a single solution cost. The algorithm halts only after verifying that no additional Pareto-optimal solution exists. Thus, we cannot stop after a given solution is found because other solutions may exist, necessitating the complete exhaustion of OPEN to assure that no unexplored Pareto-optimal paths remain.

In MOS, every generated node must undergo at least one solution dominance check (SDC) (Steps 2 and 5). Consider a given node n in Zone D . Each such node is dominated by at least one solution node C_i . Thus, when such a node n is chosen for extraction, it must be after the extraction of the goal node C_i . Therefore, SDC against C_i will prune node n , and it will never be expanded. But, at the time of generation, if the goal C_i was not yet recognized. Then performing SDC will not help, and n will be inserted to OPEN. However, unlike SOS, those nodes from Zone D in OPEN must later be extracted (and pruned) as we explain next.

We must perform an SDC on all nodes in OPEN because they might belong to each of the four Zones A, B, C, D . An example is given in Figure 5. Node n in Zone A is being expanded, even though a solution of cost C_2 has already been found. Five successors of n are generated in the different zones and in the following order: d_1 is generated, but a DC realizes that it is dominated by C_2 and thus d_1 is pruned. Then, d_2 is generated. But, since C_3 was not yet found, d_2 is not dominated by any existing solution and, thus, d_2 is inserted to OPEN. Then, the three nodes a, b , and c are generated and not pruned by any existing solution. Thus, they are all inserted into OPEN. Nodes a and b will be both extracted and expanded. Node c must be extracted with any OF before d_2 because it dominates it. Since c is a goal node, the solution of C_3 will be added to *POF*. Next, d_2 is extracted and immediately pruned by the solution of C_3 . This phenomenon (of nodes from Zone D being inserted into OPEN only to be pruned upon extraction) is observed for all OFs, including Lex.

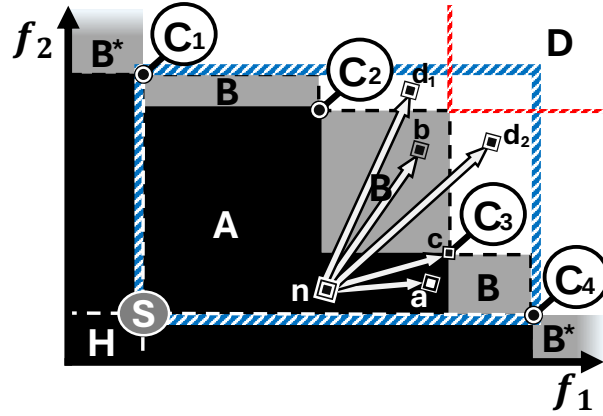


Fig. 5. Different zones in a BOS instance. Every point in the figure is a two-dimensional cost from *start* to *goal*.

6.3 Active Region Within Zone D.

When using the single-point heuristic, the *extreme solutions* (e.g., C_1 and C_4 in Figure 5) can be efficiently computed with an SOS algorithm in a preprocessing phase (Skyler, Atzmon, Felner, Salzman, Zhang, et al. 2022). This partitions the search space into the two zones according to the Blue frame in Figure 5:

(1) **Inactive Region (outside the frame (C_1, D, C_4, S)).** Recall that S is the f -value of *start*. Thus, no nodes to its left or below it will ever be generated. If a node is generated above or right of the frame, it is immediately pruned because it is clearly dominated by either C_1 or C_4 . A simple, constant-time DC against C_1 and C_4 will yield this without requiring a more advanced DC. Note that regions B^* can be pruned this way.

(2) **Active Region (inside the frame (C_1, D, C_4, S)).** These nodes must undergo the regular DC and may be inserted to OPEN and later extracted during the search. If they are in Zones A and B, they will also be expanded.

As discussed above, all OFs expand exactly the same set of nodes. Naturally, all of their neighbors are generated and, thus, all OFs generate the same set of nodes. However, OFs affect the *order* by which nodes are expanded and, thus, the order and rate by which solutions are discovered. This may be crucial in two ways. First, in an *anytime scenario*, where one needs many solutions as fast as possible. In addition, different OFs may treat generated nodes differently. Second, assume that a node n is generated in Zone D, which is dominated by a solution in C_i . Some OFs might already know about the solution in C_i and prune it upon generation without inserting it into OPEN. Other OFs might not yet know about C_i , insert node n into OPEN, then extract it and prune it later due to C_i . Therefore, the number of *extracted* nodes varies with the choice of OF, and this might affect the running time as a node in Zone D that is generated, but skips OPEN incurs less overhead than a node in Zone D that is inserted and extracted from OPEN.

Overall, understanding the interplay between expansions and extractions is critical for evaluating the performance of BOS algorithms. In scenarios that demand fast discovery of multiple solutions (e.g., anytime settings), an OF that minimizes unnecessary extractions might speed up the search.

7 Constant Time Dominance Checks in BOS

As mentioned above, there are two types of DC for node n : (1) *Path Dominance Check* (PDC), where we match n against other nodes with the same state, and (2) *Solution Dominance Check* (SDC), where we match n against existing solution costs. DC is expensive because a node n is usually matched against a possibly very large list.

Algorithm 2: Lex with constant time DC

```

1 initialize_values ()
2   for  $s \in S$  do
3      $g_2^{min}(s) \leftarrow \infty$ 
4 dominance_check (Node  $n$ )
5   if  $f_2(n) \geq g_2^{min}(goal)$  then
6     return True // SDC
7   if  $g_2(n) \geq g_2^{min}(s(n))$  then
8     return True // PDC
9   return False // non-dominated
10 update_values (Node  $n$ )
11    $g_2^{min}(s(n)) \leftarrow g_2(n)$ 

```

Hernández, Yeoh, J. A. Baier, et al. 2023 presented the BOA* algorithm, a variant of MOS-A*, that performs DC for Lex in constant time rather than linear time, for the Bi-Objective Search case of $d = 2$ (Bi-Objective A*). We next introduce constant-time DC (CTDC) for Min and Max, thereby making them competitive with Lex.

For each ordering function (OF), we maintain per-state and global parameters so that any later occurrence of a state should be only compared against these parameters to allow a constant time dominance check. The constant-time dominance check (CTDC) for any OF is divided into three parts:

- (1) initialize_values, in which we initialize the values of the parameters needed for performing the DC.
- (2) update_values, in which we update these values.
- (3) dominance_check, in which we use these values to prune nodes.

In the following subsections, we instantiate this scheme for specific OFs: for Lex in Section 7.1, for Min in Section 7.3, and for Max in Section 7.5.

7.1 Lex Constant Time DCs

We describe the constant time DC for Lex1 (See Alg. 2 whose functions should be used inside the general algorithm provided in Alg. 1 above). Lex2 is symmetrically defined by switching the objectives. A key observation (Hernández, Yeoh, J. A. Baier, et al. 2023) is that due to the lexicographic order of Lex1, g_1 of a state s when expanded via a different node again must increase. So, in order to be mutually non-dominated and not pruned, the value of g_2 must monotonically decrease (otherwise, both g_1 and g_2 will increase and the new occurrence is dominated).

Following the general three-part scheme introduced at the beginning of this section, we now instantiate initialize_values, update_values, and dominance_check for Lex.

initialize_values: To perform a constant time DC in Lex1 (Alg. 2), we maintain a g_2^{min} value for each state s . $g_2^{min}(s)$ stores the minimal g_2 value seen for s among the different nodes n that have s as their state. $g_2^{min}(s)$ is initialized to ∞ (Lines 1-3).

dominance_check: Let n be the currently explored node. SDC returns *True* if $f_2(n) \geq g_2^{min}(goal)$, i.e., the path through n cannot be developed into a path that is non-dominated by all of the already found goals (Lines 5-6). PDC returns *True* if $g_2(n) \geq g_2^{min}(s(n))$, i.e., a prior path was found that dominates the path of n (Lines 7-8).

update_values: If node n is not pruned, then $g_2^{min}(s(n))$ is updated with $g_2(n)$ (Line 11).

Example of PDC for Lex1: Let (1, 9), (3, 4), (5, 7), (6, 6), (7, 5), (8, 2) be pairs of costs of nodes (paths) leading to the same state s . These costs are presented in the order by which they are explored by Lex1. $g_2^{min}(s(n))$ is initialized

to ∞ . For (1, 9), `dominance_check` returns *False* and g_2^{min} is set to 9. Then, for (3, 4), again `dominance_check` returns *False* and $g_2^{min} \leftarrow 4$. For (5, 7), (6, 6), and (7, 5), it holds that $g_2(n) \geq g_2^{min}(s)$, they are pruned, and *True* is returned (Lines 7-8). Lastly, for (8, 2), `dominance_check` returns *False* and $g_2^{min} \leftarrow 2$. SDC is done in a similar manner, except that for each node n we now compare $f_2(n)$ with $g_2^{min}(goal)$.

7.2 Moving From g-values to f-values for Min and Max

Since Lex1 progresses along dimension 1 (see Fig. 4b), it only requires maintaining a single value (g_2^{min}) to perform constant time DC. Importantly, while nodes are expanded based on their **f**-values, in Lex, it suffices to maintain and compare **g**-values for performing DC. In fact, it is possible to maintain (f_2^{min}) and compare **f**-values (instead of **g**-values) for the DC of Lex (Alg. 2). But, it will have no impact because, in PDC, a state always has the same **h**-value and, in SDC, $\mathbf{h}(goal) = (0, 0)$. Thus, the order by which nodes are visited and whether a node is pruned or not will not be affected.

By contrast, Min and Max progress along both dimensions (see Figs. 4e and 4f). To deal with this, we classify nodes n into two classes: Class L_1 where $f_1(n) < f_2(n)$ and class L_2 where $f_1(n) > f_2(n)$ (nodes with $f_1(n) = f_2(n)$ can be in either class). Thus, for Min and Max, we need to maintain two values, one for each class. However, with these classifications, we cannot maintain **g**-values to prune nodes while also using **f**-values to prioritize nodes. The reason is that a node can be classified to L_1 based on $\mathbf{g}(n)$ but to L_2 based on $\mathbf{f}(n)$. For example, consider two nodes p and q with $\mathbf{h}(s(p)) = \mathbf{h}(s(q)) = (1, 100)$, $\mathbf{g}(p) = (1, 2)$, and $\mathbf{g}(q) = (2, 1)$. Both nodes are classified to L_1 by their **f**-values, but q would belong to L_2 based on **g**-values due to the heuristic asymmetry. To address this, we propose, for Min and Max, storing f_1^{min} and f_2^{min} (and not g_i^{min} as in Lex) and then pruning nodes based on their **f**-values. This ensures a consistent classification of nodes since the pruning and prioritization are both based on **f**-values.

7.3 Min Constant Time Dominance Checks

Alg. 3 presents the constant time DC functions for Min that should be used inside the general algorithm presented in Alg. 1.

initialize_values: Min selects the node whose $\min(f_1, f_2)$ is smallest. Hence each state keeps two minima, f_1^{min} and f_2^{min} , both initialized to ∞ (Line 3). The *Done* flag (Blue Lines: 4, 6-13) will be discussed below and should be ignored for now.

dominance_check: SDC (and PDC) are in Lines 14-17. Given node n its f_1 is compared with $f_1^{min}(goal)$ (and $f_1^{min}(s(n))$ resp.), and f_2 is compared with $f_2^{min}(goal)$ (and $f_2^{min}(s(n))$ resp.). If one of these inequalities holds, then the node is pruned. We prove the correctness of this rule next.

THEOREM 5. *Assume that node n is chosen for expansion. There exists another node m such that $s(m) = s(n)$ and $m \prec n$, iff the conditions in Line 16 prune n (for PDC).*

PROOF. Direction 1 $\Leftarrow$: Assume without loss of generality n belongs to L_1 (i.e., $(f_1(n) \leq f_2(n))$). There are now two cases.

Case 1: Let m be the node from L_1 that triggered the value of $f_2^{min}(s(n))$ (recall that $s(n) = s(m)$). Assume that the condition on the right side of Line 16 holds. This means that $f_2(m) \leq f_2(n)$. Now, since m belongs to L_1 and since it was expanded before n , it means that $f_1(m) \leq f_1(n)$. Thus, $m \prec n$.

Case 2: Let m be the node from L_2 that triggered the value of $f_1^{min}(s(n))$. Assume that the condition in the left side of Line 16 holds. This means that $f_1(m) \leq f_1(n)$.

Now observe that $f_2(n) \geq f_1(n)$ ($n \in L_1$)
 $\geq f_1(m)$ (Line 16)
 $\geq f_2(m)$ ($m \in L_2$). Therefore, $m \prec n$.

Algorithm 3: Min with constant time DC

```

1 initialize_values ()
2   for  $s \in S$  do
3      $f_1^{min}(s) \leftarrow \infty; f_2^{min}(s) \leftarrow \infty$ 
4      $Done(s) \leftarrow False$ 
5 dominance_check (Node  $n$ )
6   if  $Done(s(n))$  then
7     return True
8   if  $\min(f_1(n), f_2(n)) \geq \min(f_1^{min}(goal), f_2^{min}(goal))$  then
9      $Done(goal) \leftarrow True$  // global Done
10    return True
11  if  $\min(f_1(n), f_2(n)) \geq \min(f_1^{min}(s(n)), f_2^{min}(s(n)))$  then
12     $Done(s(n)) \leftarrow True$  // local Done
13    return True
14  if  $f_1(n) \geq f_1^{min}(goal) \parallel f_2(n) \geq f_2^{min}(goal)$  then
15    return True // SDC
16  if  $f_1(n) \geq f_1^{min}(s(n)) \parallel f_2(n) \geq f_2^{min}(s(n))$  then
17    return True // PDC
18  return False // non-dominated
19 update_values (Node  $n$ )
20  if  $f_1(n) \leq f_2(n)$  then
21     $f_2^{min}(s(n)) \leftarrow f_2(n)$ 
22  if  $f_1(n) \geq f_2(n)$  then
23     $f_1^{min}(s(n)) \leftarrow f_1(n)$ 

```

Direction 2 $\Rightarrow$: If the two m nodes above did not prune node n then no other node can prune it because their f_2 (f_1) is the minimal such value.

Identical proofs are for the case that $n \in L_2$ and for SDC. $\square$

update_values: If the current node was not pruned by the expansion's DC, update the appropriate minimum: L_1 nodes ($f_1 < f_2$) update f_2^{min} (Lines 20-21); L_2 nodes update f_1^{min} (Lines 22-23).

Example of PDC: Let (1, 9), (8, 2), (3, 4), (5, 7), (7, 5), (6, 6) be a sequence of $\mathbf{f}$ -costs of seen for the same state s . These costs are presented in the order in which Min explores them. For (1, 9), which belongs to L_1 , dominance_check returns *False* and f_2^{min} is set to 9. Then, for (8, 2), which belongs to L_2 , again dominance_check returns *False* and $f_1^{min} \leftarrow 8$. For (3, 4), dominance_check also returns *False* and $f_2^{min} \leftarrow 4$. For (5, 7), (7, 5), and (6, 6), it holds that $f_2(n) \geq f_2^{min}$, they are pruned, and *True* is returned (Lines 16-17).

7.4 Early Termination for Min

We next present an optimization that can immediately prune future nodes without performing an additional DC test. The mechanism is controlled by a Boolean flag $Done(s)$ for each state s . Its treatment in Alg. 3 is marked in Blue. Importantly, these Blue lines are executed *only* in the DC call performed *after extraction*, but not in the generation-time DC check (see Sec. 6.1). Hence, this mechanism does not affect node insertion into OPEN; it only affects whether later extracted nodes still need to be examined for DC.

This mechanism is performed as follows. In addition to the two per-state values $f_1^{min}(s)$ and $f_2^{min}(s)$, the algorithm maintains a Boolean flag $Done(s)$, initialized to *False* (Line 4). We define $m(s) = \min(f_1^{min}(s), f_2^{min}(s))$. Intuitively, $m(s)$ is the currently certified threshold for state s under Min. When the post-extraction DC call establishes that every future extracted node of s must have Min-value $\geq m(s)$, the algorithm sets $Done(s) \leftarrow True$ (Lines 9, 12). From that point on, any later extracted node n with $s(n) = s$ can be discarded immediately (Lines 11–13), without re-running the full DC procedure.

The correctness of this optimization follows from the monotonicity of extraction under Min. Specifically, the value $\min(f_1, f_2)$ of extracted nodes is non-decreasing. Therefore, once the algorithm reaches an extracted node of state s whose Min-value is already $\geq m(s)$, all subsequent extracted nodes of the same state must also be $\geq m(s)$. By construction of Alg. 3, this means that these later nodes fall entirely inside the region already known to be dominated for s . Consequently, after $Done(s)$ becomes *True*, no future node of s can survive DC.

Example: Assume the same sequence of values for node s as above: (1, 9), (8, 2), (3, 4), (5, 7), (7, 5), (6, 6). After processing (3, 4), the maintained values are $f_1^{min}(s) = 8$ and $f_2^{min}(s) = 4$, and thus $m(s) = 4$. When (5, 7) is later examined, we have $\min(5, 7) = 5 \geq 4 = m(s)$. At this point, the algorithm can conclude that state s has crossed the certified Min threshold, and it sets $Done(s) \leftarrow True$ (Line 12). Any later node of that same state, such as (7, 5) or (6, 6), is then pruned immediately using the *Done* test (Lines 11–13).

This optimization should be viewed as a strong sufficient condition. A regular DC test may discard a node as soon as that node is dominated. In contrast, setting $Done(s) \leftarrow True$ requires a stronger conclusion: not only is the current node discardable, but the maintained information is already sufficient to rule out *all future extracted nodes* of the same state.

Practically, this condition may be met only rarely, and often only after several nodes of the state have already been processed. Nevertheless, when it does hold, it eliminates all remaining DC work for that state.

A variant of the same idea, but for the solution cost, appears in Line 8. Let

$$m(goal) = \min(f_1^{min}(goal), f_2^{min}(goal)).$$

If, for an extracted node n , $\min(f_1(n), f_2(n)) \geq m(goal)$, then every node still remaining in OPEN has Min-value $\geq m(goal)$ as well, and is therefore dominated by an already found goal solution. In this case, the algorithm can safely discard the remainder of OPEN and terminate even when $OPEN \neq \emptyset$. Equivalently, the main loop of Alg. 1 can be re-written to continue while $(OPEN \neq \emptyset) \wedge \neg Done(goal)$. Thus, the search stops when either $OPEN = \emptyset$ or $Done(goal) = True$. As with the per-state version, this global stopping rule is expected to trigger only rarely, but when it does, it yields a sound early termination of the entire search.

We implemented this early-termination method using the *Done* flag. In our experiments (not shown), this method reduced the number of nodes expanded by roughly 3%. Thus, this idea is mostly theoretical.

7.5 Max Constant Time Dominance Checks

There are two differences between the constant time DC of Max (Alg. 4) and Min (Alg. 3). (1) The *Done* flag is no longer applicable because Max halts after finding one of the extreme solutions. (2) The conditions for the PDC and SDC have *and* operation (Lines 5 and 7 in Alg. 4) as Max prunes nodes with values *larger than or equal to* the corresponding f^{min} values. Running examples for constant time DC for all our OFs are omitted here but are included in the Appendix.

THEOREM 6. *Let n be the node extracted for expansion. There exists a node m with the same state ($s(m) = s(n) = x$) such that $m \prec n$ iff the PDC test in Line 7 of Algorithm 4 returns *True*.*

PROOF. ($\Rightarrow$) Assume there exists a node m with the same state as node n ($s(m) = s(n) = x$) such that $m \prec n$. As $m \prec n$, node m must be expanded before n . After m is expanded, both $f_i^{min}(x) \leq f_i(m)$ for both $i = 1, 2$. Therefore, the conditions for PDC hold and Line 7 prunes n .

Algorithm 4: Max with constant time DC

```

1 initialize_values ()
2   for  $s \in S$  do
3      $f_1^{min}(s) \leftarrow \infty; f_2^{min}(s) \leftarrow \infty$ 
4 dominance_check (Node  $n$ )
5   if  $f_1(n) \geq f_1^{min}(goal) \ \& \ f_2(n) \geq f_2^{min}(goal)$  then
6     return True // SDC
7   if  $f_1(n) \geq f_1^{min}(s(n)) \ \& \ f_2(n) \geq f_2^{min}(s(n))$  then
8     return True // PDC
9   return False // non-dominated
10 update_values (Node  $n$ )
11   if  $f_1(n) < f_1^{min}(s(n))$  then
12      $f_1^{min}(s(n)) \leftarrow f_1(n)$ 
13   if  $f_2(n) < f_2^{min}(s(n))$  then
14      $f_2^{min}(s(n)) \leftarrow f_2(n)$ 

```

($\Leftarrow$) Assume Line 7 returns *True*. WLOG, $f_1(n) \leq f_2(n)$. Let m be the node that updated $f_2^{min}(x) = f_2(m)$. Due to Line 7, $f_2(m) \leq f_2(n)$. Also, as m was already expanded and because Max extracts nodes in non-decreasing order of $M(\cdot) = \max(f_1(\cdot), f_2(\cdot))$, we have $M(m) \leq M(n)$. Because $M(n) = f_1(n)$, $f_1(m) \leq f_1(n)$. Therefore, $m \prec n$. $\square$

8 Empirical Evaluation

This section provides an empirical evaluation of our new contributions in this paper. We first study the effect of our ordering functions (OFs) on the course of developing the Pareto-optimal frontier with particular interest in the number of node expansions across the expansion phases defined above. We then compare the CTDC of the different OF.

8.1 Bi-Objective Search Ordering Functions

We first empirically examine how each OF distributes node expansions across the three expansion phases in BOS, using the share of expansions per phase as an indicator of solution-discovery pace and pruning effectiveness. Recall that, in Section 5, we partition the execution into three phases: (1) *First*, nodes expanded before the first Pareto-optimal solution is found; (2) *Mid*, nodes expanded between the first and last solution discoveries; and (3) *Last*, nodes expanded after the last solution (the proof-of-completeness phase).

We experimented on 200 random instances of the BAY road-map (DIMACS 2006) with two objectives ($d = 2$), time (obj_1) and distance (obj_2), and four objectives ($d = 4$), which added the money cost of traversing the edges (obj_3) as well as a uniform edge cost of 1 (obj_4). Table 1 shows the ratio (in %) of expansions in each of the three phases compared to the entire execution. For $d = 2$, in the First phase, as expected, Lex1 and Lex2 have a small percentage. This is because they find the (lexicographically) first solution with a perfect heuristic in the first dimension. Min alternates between Lex1 and Lex2, and finds the first solution after expanding slightly less than the sum of the two Lex orderings as it expands their intersection only once. Finally, Avg and Max expanded almost 50% of the nodes in the First phase. The Mid phase is the heaviest among all phases as it finds *all* solutions except the first one. In the Last phase, the nodes are expanded to prove that there are no more solutions in the POF. Min expanded 3.5% of the nodes after the last solution was found (black zone in Figure 4). This means that Min found

Phase	$d = 2$			$d = 4$		
	First	Mid	Last	First	Mid	Last
Lex1	17.2	82.8	0.0	9.9	90.1	0.0
Lex2	17.1	82.9	0.0	9.5	90.1	0.4
Avg	47.1	51.4	1.5	19.6	80.1	0.3
Min	27.9	68.6	3.5	11.0	86.5	2.5
Max	43.1	56.9	0.0	20.2	79.8	0.0

Table 1. Percentage of expansions in each search stage

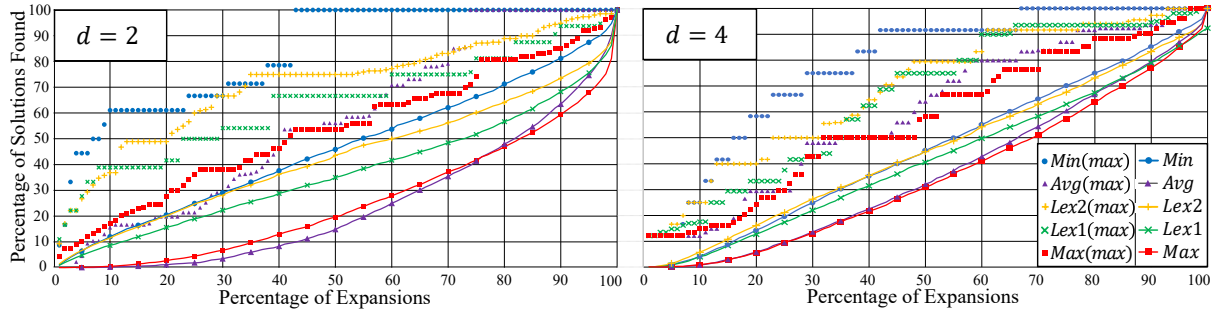


Fig. 6. Percentage of solutions found vs. percentage of expansions averaged over 200 instances

the entire *POF* the fastest, as all functions expanded the same set of nodes in each instance. Importantly, Lex and Max do not expand any node in phase Last, and the last solution found is the last node expanded for these OFs.

Figure 6(left) presents another view of this experiment by showing the average ratio of solutions found from the *POF* (*y*-axis) as a function of the progress of the expansions (*x*-axis), measured in percentiles for BOS problems ($d = 2$). There is a (full) curve for each of the OFs described above (Lex1, Lex2, Min, Avg, Max). Being close to the top-left corner is desirable because it indicates that many solutions are found quickly during the search process. The results show that Max and Avg performed worst, followed by Lex1 and Lex2, while Min performed best. This is reasonable as many completeness nodes (Zone B) are often located in the center of the *POF* (can also be seen in Figure 1), which is the last area Min explores. Additionally, the dots at the top of the figure present the maximal value (among the different runs) that the OF achieved for each percentage of expansions ($Lex_i(max)$, $Min(max)$, $Avg(max)$, $Max(max)$). The curves of the maximum values correlate with the curves of the average results in their internal order (Min is best, etc.). As mentioned, the last node explored by Lex and Max is a solution. Therefore, these functions reach 100% of the solutions in the end after expanding all nodes. By contrast, there was an instance in which Avg found all *POF* after 73% of the expansions, and Min found the entire *POF* after only 43% of the expansions. To summarize, while Lex reaches the first solution the fastest, other functions (Avg and Min) can find the entire *POF* faster, with fewer expansions.

8.2 Multi-Objective Search Experiments

Figure 6(right) shows the results for $d = 4$. The same trends reported above for BOS ($d = 2$) are seen here. That is, Min tends to perform best and finds most solutions relatively quickly. However, unlike the BOS case, with three or more objectives, the last node explored by Lex and Max is not necessarily a solution as Zone B^* still needs to be explored. Therefore, Lex2 reached 100% of the solutions after expanding 93% of the nodes, Avg after 97%, and

Lex1 after 99% (possibly in different instances). In addition, there was a case in which Avg found all *POF* after 93% of the expansions and Min found all solutions after only 67%.

8.3 Deeper Treatment and CTDC Comparison

The main advantage of BOA* (Hernández, Yeoh, J. A. Baier, et al. 2023) is the exploitation of constant-time dominance checks (CTDC) for Lex. Improvements of up to a few orders of magnitude in the CPU time were observed by the most intelligent methods that use Lex and with CTDC. Thus, without our contribution of applying CTDC to Min and Max these OFs were not practically competitive to Lex, and the value of the results in the previous section (of nodes expanded) were only theoretical and pedagogical.

We next aim to evaluate whether CTDC reduces total runtime for Min and Max relative to the classic linear-time dominance-check approach, and whether CTDC's performance makes these OFs practically competitive with Lex in CPU overhead. To this end, we ran each OF on 100 randomly sampled instances from the BAY, New York, and Colorado road maps in the DIMACS Challenge (DIMACS 2006), using the standard two-objective setting (time and distance).

Table 2 compares the total runtime of Min and Max under linear-time DC versus CTDC, averaged over the 100 instances per map. Consistent with the results for Lex (Hernández, Yeoh, J. A. Baier, et al. 2023), CTDC yields speedups of roughly two orders of magnitude, rendering Min and Max practically competitive with Lex.

Table 3 presents the average number of different operations performed over 575 instances of the BAY map. As we proved, the OF does not affect the number of expansions. Indeed, the number of expansions was 123,541 for *all* OFs. For each OF, we also report the number of extractions, DCs, percolations of OPEN (the number of shifts up or down one of the nodes in OPEN's heap), comparisons, and runtime (in seconds). The first notable result in the table is that indeed, for all OFs, more than 20,000 nodes were extracted and not expanded. This confirms our observation in Section 6.2 about the extraction of nodes. As discussed in Section 6.1, the order by which nodes are generated and solutions are found varies across the OFs, and this has an effect on the number of extractions as well as the other operations that are performed, which also vary. This is because a given node n can be pruned before it is inserted to OPEN (Step 2) by one OF, but it is only pruned after extraction (Step 5) by another OF. The number of DCs also varies because some nodes went through one DC (and were pruned at Step 2), while others went through two DCs (and were pruned at Step 5). Nevertheless, it is important to note that, while these numbers vary, the table shows they are relatively close to each other and all within 6%. The only exception is that the number of comparisons of Min and Max is almost twice than those of Lex. This is because we store both f_1^{min} and f_2^{min} rather than just one value, as we do with Lex. But, the most important result from Table 3 is that the runtime of the different OFs (until the entire *POF* was found) is also very close to each other, and they are all comparable in this regard. This is due to the fact that, now, all OFs can perform DC in a constant time.

8.4 Pace of Finding Solutions

Fig. 7(left) presents another view of this experiment. It is similar to Figure 6(left) but on the same instances reported in Table 3. It again shows the percentage of solutions found from the *POF* (y -axis) as a function of the expansion ratio (x -axis), measured in percentiles, averaged across all our cases. Exactly the same trends are

Time	BAY		NY		COL	
	Min	Max	Min	Max	Min	Max
Linear	61	59	5,189	4,567	7,068	6,338
Constant	≤ 1	≤ 1	32	31	42	42

Table 2. Runtime comparison (in seconds) between linear-time and constant-time DCs.

OF	Expand	Extract	DCs	Percolate	Compare	Time
Lex1	123,541	156,554	493,157	2,679,386	954,897	54.06
Lex2	123,541	146,828	483,430	2,232,956	926,095	53.11
Max	123,541	155,081	491,683	2,729,033	1,834,796	51.86
Min	123,541	147,784	484,386	2,588,078	1,684,661	53.35

Table 3. Detailed performance metrics on BAY benchmark

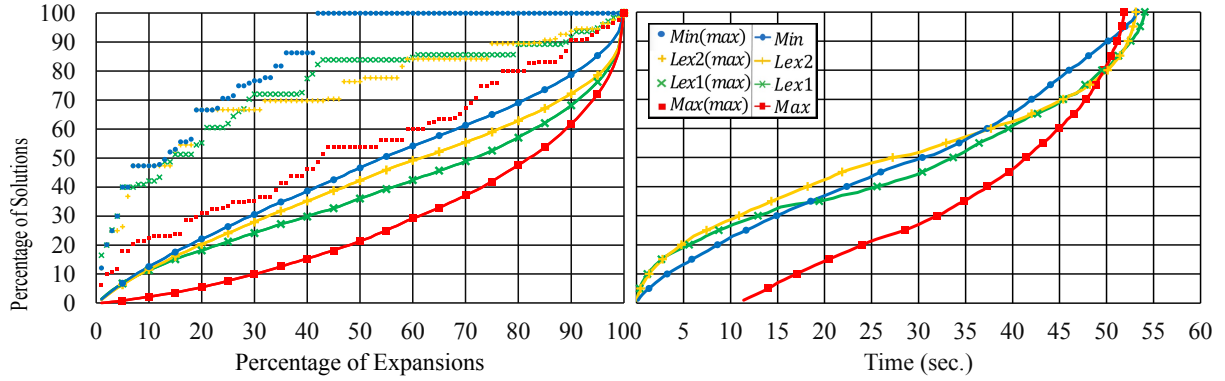


Fig. 7. Left: Solutions found vs. percentage of expansions. Right: Solutions found vs. time (sec.).

reported here, where Min is best, then the Lex variants and then Max. Fig. 7(right) shows the same experiment, but here, the x -axis is the time elapsed. Indeed, since all algorithms perform DC in constant time, the same trends as in Fig. 7(left) are observed. But, there is a slight shift in runtime (compared to the number of expansions) of Min and Max of $\approx 6\%$ compared to Lex. Here, either Min and Lex2 take the lead with a strong advantage to Min in the second half of the process, towards the end. The difference between the two Lex curves stems from their different objectives and relevant Pareto-optimal frontier solutions. Thus, in time too, Min is performing very well, best in the second half, and is the most robust variant.

9 Practical Comparison of the OFs

Based on the above, there is no significant advantage in the runtime of any OF (for recognizing and proving the *POF*), and they are all comparable due to the fact that we have also presented a method to perform constant time DC for Min and Max for BOS. We thus next cover the pros and cons of each OF, and it is left for the user to decide which OF is preferable for the given circumstance.

Lexicographic Ordering (Lex). Lex prioritizes one objective and uses the other to break ties. In BOS, two variants exist: Lex1 (prioritizing f_1) and Lex2 (prioritizing f_2). Lex is simple to implement and can quickly identify an extreme solution at one end of the *POF*, often making Lex the fastest to find the *first* solution. However, because Lex systematically proceeds from one extreme to the other, solutions at the opposite end of the *POF* are discovered late in the search. Moreover, the choice between Lex1 and Lex2 can yield different numbers of DCs, sometimes leading to notable runtime variations depending on how the two objectives scale and are prioritized.

Maximum Ordering (Max). Max selects nodes by examining the larger of the two cost components, thereby emphasizes more balanced trade-offs (i.e., solutions in the middle of the *POF*) early in the search rather than focusing on an extreme. This can be advantageous in many BOS scenarios, particularly for anytime applications

where discovering intermediate (i.e., balanced) solutions quickly is desirable. However, Max tends to delay the discovery of extreme solutions, as neither objective is explicitly favored. It may also incur a higher initial overhead by exploring a broader central region of the cost space before reaching the first solution. Although Max requires normalizing cost dimensions to prevent one objective from dominating, it is not sensitive to which dimension is labeled “first” or “second” after normalization, often resulting in more robust (than Lex) performance across different problem scales.

Minimum Ordering (Min). Min prioritizes the smaller of the two objectives and can be viewed as interleaving Lex1 and Lex2. By simultaneously favoring paths that are minimal in either dimension, Min can expand extremes of the *POF* in parallel, sometimes recognizing the entire *POF* earlier than other OFs. This behavior arises partly from enhanced pruning, as Min can more promptly detect and eliminate dominated portions of the search space. Additionally, Min incorporates a *Done* condition, allowing the search to halt as soon as no further non-dominated paths remain. Nevertheless, balanced solutions—often highly practical—may be discovered later, since Min initially focuses on extremes. As with Max, Min requires normalized cost dimensions, yet its performance does not depend on which dimension is considered first, resulting in more consistent behavior across varying objective scales.

To summarize, selecting an OF, thus, hinges on problem requirements, such as the need for early identification of certain solution types (e.g., extremes vs. balanced trade-offs) and the relative expense of normalizing multiple objectives.

10 Summary and Conclusions

We classified MOS nodes into MENs, NENs, and MBENs. In addition, we considered the issue of *OF*, which are used by MOS algorithms to decide which node to expand next based on their *f*-values. We presented several OFs and compared them experimentally. As all OFs must expand the same set of nodes, we showed the benefit of some OFs with regard to the order in which these nodes are explored.

We further developed the theory of NEN and showed that nodes inserted to OPEN must later be extracted before halting. Additionally, we introduced a constant-time DC mechanism for Min and Max in BOS, making them comparable to and competitive with Lex. Our analysis shows that, while all OFs expand the same set of nodes, they may differ in their pace of finding the solutions and the pace of pruning. We also presented an early termination condition for Min, which can reduce the extraction overhead. Experimental results on road networks indicate that Min reaches more solutions earlier than other OFs and that, using a constant time DC, all OFs share almost a similar runtime.

Possible directions for future work include (1) examining constant time DC for other OFs; (2) defining the general set of OFs for which a constant time DC exists; (3) extending these techniques to higher-dimensional MOS, which may further reduce the runtime significantly and allow early stopping by a *Done* flag; (4) further developing multi-valued heuristics and see how they correlate with the finding in this paper.

References

- S. Ahmadi, G. Tack, D. Harabor, and P. Kilby. 2021. “Bi-Objective Search with Bi-Directional A^* .” In: *29th Annual European Symposium on Algorithms (LIPIcs)*. Ed. by P. Mutzel, R. Pagh, and G. Herman. Vol. 204, 3:1–3:15. doi:[10.4230/LIPIcs.ESA.2021.3](https://doi.org/10.4230/LIPIcs.ESA.2021.3).
- D. Bachmann, F. Bökler, J. Kopeck, K. Popp, B. Schwarze, and F. Weichert. 2018. “Multi-Objective Optimisation Based Planning of Power-Line Grid Expansions.” *ISPRS International Journal of Geo-Information*, 7, 7, 258.
- J. Climaco and M. Pascoal. 2012. “Multicriteria Path and Tree Problems: Discussion on Exact Algorithms and Applications.” *ITOR*, 19, 1-2, 63–98.
- J. Current and M. Marsh. 1993. “Multiobjective Transportation Network Design and Routing Problems: Taxonomy and Annotation.” *EJOR*, 65, 1, 4–19.
- R. Dechter and J. Pearl. 1985. “Generalized Best-First Search Strategies and the Optimality of A^* .” *J. ACM*, 32, 3, 505–536. doi:[10.1145/3828.3830](https://doi.org/10.1145/3828.3830).
- DIMACS. 2006. *The 9th DIMACS Implementation Challenge: Shortest Path*. (2006). <http://users.diag.uniroma1.it/challenge9/download.shtml>.

- J. E. Doran and D. Michie. 1966. "Experiments with the Graph Traverser Program." *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences*, 294, 1437, 235–259.
- J. Eckerle, J. Chen, N. R. Sturtevant, S. Zilles, and R. C. Holte. 2017. "Sufficient Conditions for Node Expansion in Bidirectional Heuristic Search." In: *ICAPS*, 79–87.
- M. Emmerich and A. Deutz. 2018. "A tutorial on multiobjective optimization: fundamentals and evolutionary methods." *Natural computing*, 17, 3, 585–609.
- M. Fu, A. Kuntz, O. Salzman, and R. Alterovitz. 2019. "Toward Asymptotically-Optimal Inspection Planning Via Efficient Near-Optimal Graph Search." In: *Robotics: Science and Systems*.
- M. Fu, O. Salzman, and R. Alterovitz. 2021. "Computationally-Efficient Roadmap-based Inspection Planning via Incremental Lazy Search." In: *ICRA*, 7449–7456.
- F. Geisser, P. Haslum, S. Thiébaux, and F. Trevizan. 2022. "Admissible Heuristics for Multi-Objective Planning." In: *ICAPS*, 100–109.
- M. Goldenberg, A. Felner, R. Stern, G. Sharon, N. R. Sturtevant, R. C. Holte, and J. Schaeffer. 2014. "Enhanced Partial Expansion A." *J. Artif. Intell. Res.*, 50, 141–187.
- B. Goldin and O. Salzman. 2021. "Approximate Bi-Criteria Search by Efficient Representation of Subsets of the Pareto-Optimal Frontier." In: *ICAPS*, 149–158.
- P. E. Hart, N. J. Nilsson, and B. Raphael. 1968. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE transactions on Systems Science and Cybernetics*, 4, 2, 100–107.
- C. Hernández, W. Yeoh, J. Baier, A. Felner, O. Salzman, H. Zhang, S.-H. Chan, and S. Koenig. 2023. "Multi-objective Search via Lazy and Efficient Dominance Checks." In: *IJCAI*.
- C. Hernández, W. Yeoh, J. A. Baier, H. Zhang, L. Suazo, S. Koenig, and O. Salzman. 2023. "Simple and efficient bi-objective search algorithms via fast dominance checks." *Artif. Intell.*, 314, 103807.
- C.-L. Hwang and A. S. Masud. 2012. *Multiple objective decision making—methods and applications: a state-of-the-art survey*. Vol. 164. Springer Science & Business Media.
- L. Mandow and J. L. P. De La Cruz. 2005. "A New Approach to Multi-Objective A* Search." In: *IJCAI*, 218–223.
- L. Mandow and J. Pérez-de-la-Cruz. 2010. "Multiobjective A* search with consistent heuristics." *J. ACM*, 57, 5, 27:1–27:25.
- K. Miettinen. 2012. *Nonlinear multiobjective optimization*. Vol. 12. Springer Science & Business Media.
- F. J. Pulido, L. Mandow, and J.-L. Pérez-de-la-Cruz. 2015. "Dimensionality Reduction in Multiobjective Shortest Path Search." *CAR*, 64, 60–70.
- D. M. Roijers and S. Whiteson. 2017. *Multi-Objective Decision Making*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers.
- O. Salzman, A. Felner, C. Hernández, H. Zhang, S. Chan, and S. Koenig. 2023. "Heuristic-Search Approaches for the Multi-Objective Shortest-Path Problem: Progress and Research Opportunities." In: *IJCAI-23*. ijcai.org, 6759–6768.
- A. J. Skriver. 2000. "A Classification of Bicriterion Shortest Path (BSP) Algorithms." *Asia Pacific J. of Oper. Res.*, 17, 2, 199–212.
- S. Skyler, D. Atzmon, A. Felner, O. Salzman, C. Hernandez Ulloa, and S. Koenig. 2026. "Deeper Treatment of the Bi-objective Search Framework." In: *AAAI*.
- S. Skyler, D. Atzmon, A. Felner, O. Salzman, H. Zhang, S. Koenig, W. Yeoh, and C. Hernández. 2022. "Bounded-Cost Bi-Objective Heuristic Search." In: *SoCS22*, 239–243.
- S. Skyler, S. Shperberg, et al.. 2024. "Theoretical Study on Multi-objective Heuristic Search." In: *the International Joint Conference on Artificial Intelligence (IJCAI)*, 7021–7028.
- B. S. Stewart and C. C. White III. 1991. "Multiobjective A*." *Journal of the ACM*, 38, 4, 775–814.
- Z. M. Tarapata. 2007. "Selected Multicriteria Shortest Path Problems: An Analysis of Complexity, Models and Adaptation of Standard Algorithms." *IJAM*, 17, 2.
- E. Ulungu and J. Teghem. 1991. "Multi-Objective Shortest Path Problem: A Survey." In: *Workshop on Multicriteria Decision Making*, 176–188.
- H. Zhang, O. Salzman, T. K. S. Kumar, A. Felner, S. Skyler, C. H. Ulloa, and S. Koenig. 2023. "Towards Effective Multi-Valued Heuristics for Bi-objective Shortest-Path Algorithms via Differential Heuristics." In: *SoCS*.

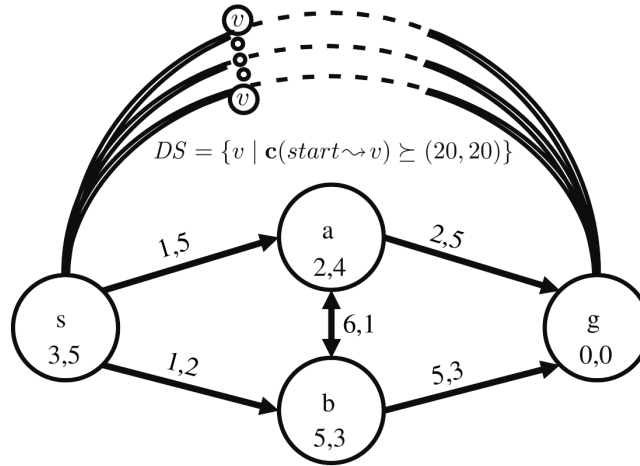


Fig. 8. Running Example Graph.

A Running Examples

Figure 8 presents a graph where a path is needed from *start* to *goal*. Note the set *DS* on the top of the figure. It includes neighbors of *start* all with $\mathbf{g}(n) \geq 20$. All nodes in the *Dominated Set* (*DS*) are generated when expanding *start* at step 1. They are being inserted to OPEN as there are not yet nodes that were expanded and dominate these nodes. We omit writing nodes of *DS* in the tables for clarity purposes.

Tables 4–7 provide running examples of executing BOS-A* on this graph with the following OFs: Lex1, Lex2, Max, and Min. Each row corresponds to the extraction of a node from OPEN and shows the OPEN queue and the current g_1^{min} , g_2^{min} values *before* the extraction of this node. For each node in OPEN we report its state, *g*-values, and *f*-values. That is, a state *x* is given in the form $\{x, \mathbf{g}(x), \mathbf{f}(x)\}$. The states are given with a track of the path from *start*. For example, b_{sa} means that we are at state *b* but the path is: $\{s, a, b\}$. A node that is struck through with a line means that this node was pruned in the DC. The node marked “←” is the one that is about to be extracted next. The right columns provide the resulting minima updates in the following template: the state *x*, $g_1^{min}(x)$, $g_2^{min}(x)$. In the case of Lex1, and Lex2 we only provide $g_2^{min}(x)$ and $g_1^{min}(x)$, respectively.

A.1 Lex1

Table 4 provides the execution trace of Lex1. Nodes are extracted in the following order: $\{start, b, goal, a, goal\}$. Then all the nodes $\{x | state(x) \in DS\}$ are also extracted (not shown in the table). At step 3, the first solution is found: g_{sa} . Note that at step 6, nodes a_{sb} and b_{sa} were extracted but pruned by the DC because both have $f_2 \geq 5$, which is the $g_2^{min}(goal)$. Then, all nodes that were generated for states in *DS* are extracted one by one and pruned by the DC (not shown in the table). Theoretically, the size of *DS* can be much larger than the four states we presented in the example. After extracting and pruning all *DS* nodes, OPEN is empty and the search halts with the *POF* of $\{goal_{sa}, goal_{sb}\}$.

A.2 Lex2

Table 5 provides the execution trace of Lex2. Nodes are extracted in the following order: $\{start, a, goal, b, goal\}$. Then all the nodes $\{x | state(x) \in DS\}$ are also extracted (not shown in the table). At step 3, the first solution is

	OPEN list $state(n), g(n), f(n)$	g_i^{min} Update		
		$state$	f_1^{min}	g_2^{min}
1	$(start, (0, 0), (3, 5)) \leftarrow$	$start$		5
2	$(a_s, (1, 5), (3, 9)) \leftarrow$ $(b_s, (1, 2), (6, 5))$	a		9
3	$(b_s, (1, 2), (6, 5))$ $(b_{sa}, (7, 6), (12, 9))$ $(goal_{sa}, (3, 10), (3, 10)) \leftarrow$	$goal$		10
4	$(b_s, (1, 2), (6, 5)) \leftarrow$ $(b_{sa}, (7, 6), (12, 9))$	b		5
5	$(b_{sa}, (7, 6), (12, 9))$ $(a_{sb}, (7, 3), (9, 7))$ $(goal_{sb}, (6, 5), (6, 5)) \leftarrow$	$goal$		5
6	$(a_{sb}, (7, 3), (9, 7)) \leftarrow^1 SDC_{exp}$ $(b_{sa}, (7, 6), (12, 9)) \leftarrow^2 SDC_{exp}$			

Table 4. Lex1

found: g_{sa} . Note that at step 4, node a_{sb} was extracted but pruned by the DC because it has $f_1 > 6$, which is the $g_1^{min}(goal)$. Also, at step 6, node b_{sa} was pruned by the DC during the node's generation because it has $f_1 \geq 6$, which is the $g_1^{min}(goal)$. Then, all nodes that were generated for states in DS are extracted one by one and pruned by the DC. After extracting and pruning all DS nodes, after expanding $goal_{sa}$ at step 6, OPEN is empty and the search halts with the POF of $\{goal_{sb}, goal_{sa}\}$.

	OPEN list $state(n), g(n), f(n)$	g_i^{min} Update		
		$state$	f_1^{min}	g_2^{min}
1	$(start, (0, 0), (3, 5)) \leftarrow$	$start$	3	
2	$(a_s, (1, 5), (3, 9))$ $(b_s, (1, 2), (6, 5)) \leftarrow$	b	6	
3	$(a_s, (1, 5), (3, 9))$ $(a_{sb}, (7, 3), (9, 7))$ $(goal_{sb}, (6, 5), (6, 5)) \leftarrow$	$goal$	6	
4	$(a_s, (1, 5), (3, 9))$ $(a_{sb}, (7, 3), (9, 7)) \leftarrow SDC_{exp}$			
5	$(a_s, (1, 5), (3, 9)) \leftarrow$	a	3	
6	$(b_{sa}, (7, 6), (12, 9)) \leftarrow SDC_{gen}$ $(goal_{sa}, (3, 10), (3, 10)) \leftarrow$	$goal$	3	

Table 5. Lex2

A.3 Max

Table 6 provides the execution trace of Max. Nodes are extracted in the following order: $\{start, b, goal, a, goal\}$. Then all the nodes $\{x | state(x) \in DS\}$ are also extracted (not shown in the table). At step 3, the first solution is

found: g_{sb} . Note that at step 5, node a_{sb} was extracted but pruned by the DC because it has $f_1 > 6$ & $f_2 > 5$, which are the $g_1^{min}(goal)$ and $g_2^{min}(goal)$. Also at step 5, node b_{sa} was pruned by the DC during the node's generation because it has $f_1 > 6$ & $f_2 > 5$, which are the $g_1^{min}(goal)$ and $g_2^{min}(goal)$. Then, all nodes that were generated for states in DS are extracted one by one and pruned by the DC. After extracting and pruning all DS nodes, after expanding $goal_{sa}$ at step 6, OPEN is empty and the search halts with the POF of $\{goal_{sb}, goal_{sa}\}$.

	OPEN list $state(n), g(n), f(n)$	g_i^{min} Update		
		$state$	f_1^{min}	g_2^{min}
1	$(start, (0, 0), (3, 5)) \leftarrow$	$start$	3	5
2	$(a_s, (1, 5), (3, 9))$ $(b_s, (1, 2), (6, 5)) \leftarrow$	b	6	5
3	$(a_s, (1, 5), (3, 9))$ $(a_{sb}, (7, 3), (9, 7))$ $(goal_{sb}, (6, 5), (6, 5)) \leftarrow$	$goal$	6	5
4	$(a_s, (1, 5), (3, 9)) \leftarrow$ $(a_{sb}, (7, 3), (9, 7))$	a	3	9
5	$(a_{sb}, (7, 3), (9, 7)) \leftarrow SDC_{exp}$ $(b_{sa}, (7, 6), (12, 9)) SDC_{gen}$ $(g_{sa}, (3, 10), (3, 10))$			
6	$(g_{sa}, (3, 10), (3, 10)) \leftarrow$	$goal$	3	5

Table 6. Max

A.4 Min

In Min, we also provide information on the *Done* flag once it is set to True. In Min the DC routine sets *Done(s)* once $\min(f_1, f_2)$ for state x can never improve (denoted *local Done*), and if *Done(goal)* is set *True* (denoted *global Done*), we can halt the search even if OPEN is not empty once *Done(goal) = True*. This happens for the goal in iteration 6. *Done(goal)* becomes *True* and the search terminates immediately. Thus, none of the nodes whose states lie in the dominated region $DS = \{v \mid c(start \rightsquigarrow v) \succeq (20, 20)\}$ is ever removed from OPEN. The other orderings lack this early-stop test, so after the final iteration, they still extract and discard the (dominated) *DS* nodes, as was seen in the previous subsections.

Table 7 provides the execution trace of Min. Nodes are extracted in the following order: $\{start, a, goal, b, goal\}$. At step 3, the first solution is found: g_{sa} . At step 6, the *Done* flag of *goal* is set to *True* and thus, the search halts due to *global Done*, and all the nodes that are in OPEN (*DS* nodes and b_{sa}) are never extracted. Thus, nodes in OPEN are ignored and the search halts with the POF of $\{goal_{sa}, goal_{sb}\}$ after only 6 extractions. This shows the relative strength of Min which allows early termination without fully emptying OPEN. Since the *DS* set might include a very large number of nodes this might cause a significant speedup in the search.

	OPEN list $state(n), \mathbf{g}(n), \mathbf{f}(n)$	g_i^{min} Update		
		$state$	f_1^{min}	g_2^{min}
1	$(start, (0, 0), (3, 5)) \leftarrow$	$start$	∞	5
2	$(a_s, (1, 5), (3, 9)) \leftarrow$ $(b_s, (1, 2), (6, 5))$	a	∞	9
3	$(b_s, (1, 2), (6, 5))$ $(b_{sa}, (7, 6), (12, 9))$ $(goal_{sa}, (3, 10), (3, 10)) \leftarrow$	$goal$	∞	10
4	$(b_s, (1, 2), (6, 5)) \leftarrow$ $(b_{sa}, (7, 6), (12, 9))$	b	6	∞
5	$(b_{sa}, (7, 6), (12, 9))$ $(a_{sb}, (7, 3), (9, 7))$ $(goal_{sb}, (6, 5), (6, 5)) \leftarrow$	$goal$	6	10
6	$(b_{sa}, (7, 6), (12, 9))$ $(a_{sb}, (7, 3), (9, 7)) \leftarrow$	$min f_1(a_{sb}), f_2(a_{sb}) \geq min g_1^{min}(goal), g_1^{min}(goal)$ $min(9, 7) = 7 \geq 6 = min(6, 10)$ $Done(goal) \leftarrow True$		

Table 7. Min

B Online Resources

The empirical evaluation uses public road-network benchmark instances from the DIMACS Challenge suite (DIMACS 2006). The experiments reported in Section 8 use the BAY, New York, and Colorado maps with the objective functions described there. No new benchmark data set is introduced in this paper.

To support reproducibility, we will make the experimental artifacts available upon publication.

C Reproducibility Checklist for JAIR

Select the answers that apply to your research – one per item.

All articles:

- (1) All claims investigated in this work are clearly stated. [yes]
- (2) Clear explanations are given how the work reported substantiates the claims. [yes]
- (3) Limitations or technical assumptions are stated clearly and explicitly. [yes]
- (4) Conceptual outlines and/or pseudo-code descriptions of the AI methods introduced in this work are provided, and important implementation details are discussed. [yes]
- (5) Motivation is provided for all design choices, including algorithms, implementation choices, parameters, data sets and experimental protocols beyond metrics. [yes]

Articles containing theoretical contributions:

Does this paper make theoretical contributions? [yes]

If yes, please complete the list below.

- (1) All assumptions and restrictions are stated clearly and formally. [yes]
- (2) All novel claims are stated formally (e.g., in theorem statements). [yes]
- (3) Proofs of all non-trivial claims are provided in sufficient detail to permit verification by readers with a reasonable degree of expertise (e.g., that expected from a PhD candidate in the same area of AI). [yes]
- (4) Complex formalism, such as definitions or proofs, is motivated and explained clearly. [yes]
- (5) The use of mathematical notation and formalism serves the purpose of enhancing clarity and precision; gratuitous use of mathematical formalism (i.e., use that does not enhance clarity or precision) is avoided. [yes]
- (6) Appropriate citations are given for all non-trivial theoretical tools and techniques. [yes]

Articles reporting on computational experiments:

Does this paper include computational experiments? [yes]

If yes, please complete the list below.

- (1) All source code required for conducting experiments is included in an online appendix or will be made publicly available upon publication of the paper. The online appendix follows best practices for source code readability and documentation as well as for long-term accessibility. [partially]
- (2) The source code comes with a license that allows free usage for reproducibility purposes. [partially]
- (3) The source code comes with a license that allows free usage for research purposes in general. [partially]
- (4) Raw, unaggregated data from all experiments is included in an online appendix or will be made publicly available upon publication of the paper. The online appendix follows best practices for long-term accessibility. [partially]
- (5) The unaggregated data comes with a license that allows free usage for reproducibility purposes. [partially]
- (6) The unaggregated data comes with a license that allows free usage for research purposes in general. [partially]

- (7) If an algorithm depends on randomness, then the method used for generating random numbers and for setting seeds is described in a way sufficient to allow replication of results. [partially]
- (8) The execution environment for experiments, the computing infrastructure (hardware and software) used for running them, is described, including GPU/CPU makes and models; amount of memory (cache and RAM); make and version of operating system; names and versions of relevant software libraries and frameworks. [partially]
- (9) The evaluation metrics used in experiments are clearly explained and their choice is explicitly motivated. [yes]
- (10) The number of algorithm runs used to compute each result is reported. [yes]
- (11) Reported results have not been “cherry-picked” by silently ignoring unsuccessful or unsatisfactory experiments. [yes]
- (12) Analysis of results goes beyond single-dimensional summaries of performance (e.g., average, median) to include measures of variation, confidence, or other distributional information. [no]
- (13) All (hyper-) parameter settings for the algorithms/methods used in experiments have been reported, along with the rationale or method for determining them. [NA]
- (14) The number and range of (hyper-) parameter settings explored prior to conducting final experiments have been indicated, along with the effort spent on (hyper-) parameter optimisation. [NA]
- (15) Appropriately chosen statistical hypothesis tests are used to establish statistical significance in the presence of noise effects. [NA]

Articles using data sets:

Does this work rely on one or more data sets (possibly obtained from a benchmark generator or similar software artifact)? [yes]

If yes, please complete the list below.

- (1) All newly introduced data sets are included in an online appendix or will be made publicly available upon publication of the paper. The online appendix follows best practices for long-term accessibility with a license that allows free usage for research purposes. [NA]
- (2) The newly introduced data set comes with a license that allows free usage for reproducibility purposes. [NA]
- (3) The newly introduced data set comes with a license that allows free usage for research purposes in general. [NA]
- (4) All data sets drawn from the literature or other public sources (potentially including authors’ own previously published work) are accompanied by appropriate citations. [yes]
- (5) All data sets drawn from the existing literature (potentially including authors’ own previously published work) are publicly available. [yes]
- (6) All new data sets and data sets that are not publicly available are described in detail, including relevant statistics, the data collection process and annotation process if relevant. [NA]
- (7) All methods used for preprocessing, augmenting, batching or splitting data sets (e.g., in the context of hold-out or cross-validation) are described in detail. [partially]

Explanations on any of the answers above (optional):

The paper is primarily theoretical and uses computational experiments to illustrate and evaluate the proposed ordering functions and constant-time dominance-check implementations. The benchmark maps are public DIMACS road-network instances, and no new data set is introduced. Items concerning source code, raw logs, licenses, random seeds, and hardware are marked “partially” because the manuscript states the experimental setup and

reported instance counts but does not yet include a permanent public repository, final license statement, random seed list, or full machine specification. Items concerning hyper-parameter search and statistical significance testing are marked “NA” because the evaluated algorithms do not involve tuned hyper-parameters and the reported comparisons are deterministic algorithm runs on fixed sampled instances rather than noisy repeated trials.

Received Insert received date; accepted Insert accepted date